PROJECT REPORT

Explainable AI for Daily Short-Term Risk Prediction in Coinage Metal Markets

University of West London – School of Computing and Engineering

-

Final Year Project

Level 6

Module Code: CP60046E

Academic Year: 2025-2026

-

Name: Mohammed Adnan Osman

Student ID: 33114153

Word Count: 10817

-

Supervisor: Dr Nasim Dadashi

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr Nasim Dadashi, for her continued guidance, support, and constructive feedback throughout this project. Her expertise and encouragement have been invaluable in shaping both the direction and quality of this research.

I would also like to thank the University of West London School of Computing and Engineering for providing the academic framework and resources that made this project possible.

Abstract

Machine learning can improve financial risk analysis, but many models are still difficult to interpret. This project addresses that issue by developing an explainable machine learning system to predict downside risk events in the gold, silver, and copper markets, defined as daily price declines of more than 2%.

A Random Forest classifier was used to compare two models. The baseline model included technical and macroeconomic features, while the enhanced model also included FinBERT sentiment scores. Data from 2020 to 2025 were stored in a PostgreSQL database, producing 4,525 trading records. Class imbalance was handled with balanced class weights, and a chronological split was used to reduce look-ahead bias.

The enhanced gold model achieved a ROC-AUC of 0.752, compared with 0.733 for the baseline. Silver also improved, while copper showed a notable reduction in Brier Score from 0.132 to 0.087. Backtesting on 2025 data showed that the gold model generalised well, reaching a ROC-AUC of 0.847. SHAP analysis indicated that the models captured economically meaningful relationships, such as Treasury yields being important for gold predictions and S&P 500 returns being important for copper. Sentiment features were not among the most important variables, mainly because of limited coverage.

Overall, the results suggest that interpretable machine learning can support commodity risk prediction by producing both useful risk estimates and understandable explanations. Future work could improve sentiment coverage, refine risk thresholds, and test recurrent neural networks for temporal patterns.

Contents

List of Figures

List of Tables

1 Introduction

Machine learning improves financial risk analysis by finding patterns in large datasets that traditional methods miss (Fischer and Krauss, 2018). However, many high-performing models are "black boxes," which creates problems in finance where analysts must explain their predictions (Molnar, 2022).

Gold, silver, and copper are important global assets, yet short-term risk prediction in these markets receives less research attention than stocks or cryptocurrencies (Giudici et al., 2024). This project addresses three gaps:

1. Limited use of explainable AI in commodity markets.
2. Few studies combining sentiment analysis with interpretable models for these metals.
3. Emphasis on accuracy over explainability.

The system predicts daily downside risk events (price drops >2%) in gold, silver, and copper using Random Forest with FinBERT sentiment features. Research question: Can this approach deliver accurate predictions with clear, theory-aligned explanations? This balances performance and transparency needs in financial AI (Doshi-Velez and Kim, 2018).

1.1 Aim and Objectives

1.1.1 Aim

The aim of this project is to build an explainable machine learning system to predict short-term price drops in gold, silver, and copper markets. The system uses price data, economic indicators, and news sentiment to create probability-based risk scores. It uses SHAP explanations to make the model transparent and support better financial decision-making.

1.1.2 Objectives

1. Review the existing research on machine learning, explainable AI, and sentiment analysis in financial risk prediction to identify research gaps.

2. Build a database and automated pipelines to collect historical price data, economic indicators, and news sentiment from 2020 to 2025.

3. Build and test a baseline Random Forest model using only technical and economic data to predict daily price drops.

4. Develop an enhanced model incorporating FinBERT-derived sentiment features alongside technical and macroeconomic indicators and evaluate whether sentiment integration improves predictive performance relative to the baseline.

5. Use SHAP to explain both models and confirm that the results match financial theory.

6. Use metrics like Brier Score and Log Loss to ensure the risk scores are reliable for practical use.

7. Test the models on unseen data from 2025 and use SHAP to explain real-world market events.

8. Critically evaluate the results, document limitations, and provide recommendations for future work.

2 Literature Review

2.1 Artificial intelligence and Machine Learning in Financial Risk Prediction

Machine learning has become increasingly important in financial risk prediction over the past decade, largely because traditional statistical models struggle with large, complex financial datasets. Classical approaches such as ARIMA and GARCH rely on assumptions of linearity and stationarity that are often violated in real markets, where returns are non-linear, heavy-tailed, and subject to structural breaks (Fischer and Krauss, 2018). Fischer and Krauss (2018) showed that LSTM networks can outperform traditional econometric benchmarks in daily stock return prediction, but they also highlighted a major limitation: deep learning models are often difficult to interpret.

Random Forest classifiers offer a strong alternative because they combine good predictive performance with greater interpretability. By building multiple decorrelated trees and aggregating their outputs, Random Forests reduce the overfitting risk associated with single decision trees and work well with noisy, high-dimensional financial data (Peng and Yan, 2021). A review by Peng and Yan (2021) found that ensemble methods performed competitively across tasks such as credit risk, volatility forecasting, and fraud detection, while also offering a better balance between accuracy and transparency than deep neural networks.

Class imbalance is another major challenge in financial risk prediction. Adverse events such as large daily price declines occur much less often than normal events, so a model can appear accurate while still missing the cases that matter most. To address this, class-weighted training can increase the penalty for misclassifying minority risk events and improve recall without a major loss in overall performance (Peng and Yan, 2021). This approach directly informs the methodology used in this study, where balanced class

weighting is applied during Random Forest training to reduce the chance of overlooking downside risk events.

2.2 Explainable AI and Model Interpretability in Finance

Explainable AI has become important because predictive accuracy alone is not enough in high-stakes areas such as finance, healthcare, and law. Doshi-Velez and Kim (2018) argue that explanations are necessary when the problem is not fully specified and the cost of a wrong decision is high. In finance, this matters because risk assessments must be justified, aligned with economic reasoning, and consistent with regulatory expectations. As a result, black-box models are often harder to use in practice (Monica Hovsepian, 2023).

SHAP, introduced by Lundberg and Lee (2017), is one of the most widely used methods for explaining machine learning models in finance. It uses cooperative game theory to estimate each feature's contribution to a prediction and is valued because it provides both local explanations and global feature importance. A review by Danie et al. (2025) found that SHAP was the most commonly used explainability method in financial XAI research.

SHAP has also been applied in commodity forecasting. Jabeur, Mefteh-Wali and Viviani (2024) used SHAP with XGBoost for gold price prediction and found that macroeconomic variables such as the US Dollar Index, crude oil prices, and Treasury yields were the most influential features. Their work supports the view that gold is strongly driven by macroeconomic conditions, although it focuses on price forecasting rather than downside risk classification and does not include sentiment features.

Research also suggests that commodity markets deserve separate analysis. Giudici et al. (2024) found that most finance XAI studies focus on equities, while commodities receive less attention. They also showed that commodity price drivers differ from those in equity markets, with supply and industrial demand playing a larger role. Zhang, Liang and Ou (2024) similarly showed that silver and copper are shaped more by cross-metal relationships and industrial demand than gold.

SHAP is powerful, but it can be computationally expensive. For that reason, this study uses TreeExplainer, which is efficient for tree-based models such as Random Forest while remaining theoretically consistent.

2.3 Natural Language Processing and Sentiment Analysis in Market Prediction

Sentiment analysis has long been used in financial forecasting. Early research by Bollen, Mao and Zeng (2011) showed that Twitter mood could help predict movements in the Dow Jones index, suggesting that public sentiment can contain useful market information beyond price data alone.

Modern language models have improved this idea by capturing context more effectively than simple word lists. FinBERT, a model trained on financial news, earnings reports, and analyst commentary, is designed to understand finance-specific language patterns better than general-purpose BERT models (Araci, 2019; Liu et al., 2020). Studies have shown that it can outperform general BERT on financial text classification, especially when financial jargon is involved (Huang, Wang and Yang, 2022).

Recent work also suggests that FinBERT features can improve forecasting models. Gu et al. (2024) found that adding FinBERT sentiment to an LSTM improved accuracy over price data alone. Another study reported that a FinBERT-XGBoost model with SHAP explanations outperformed both a technical baseline and a lexicon-based sentiment model, with an AUC of 0.748 compared with 0.593 for the technical baseline (Mathematics, 2025). These results suggest that sentiment can be especially valuable during volatile periods, when it may capture fear and uncertainty more effectively than price-based indicators.

However, the value of sentiment is not universal. Other studies found that its impact depends on the asset class, data source, and prediction horizon (Li et al., 2020), and that FinBERT is not always better than simpler methods (Liapis and Karanikola, 2023). For this reason, this study compares a baseline model without sentiment against an enhanced model with FinBERT to test whether sentiment improves coinage metal risk prediction.

The three metals in this study also behave differently. Gold is mainly influenced by macroeconomic and geopolitical factors, silver has both monetary and industrial roles, and copper is closely linked to global growth and equity markets. This makes the task more challenging than studies focused on a single asset, since the model must handle three distinct risk patterns.

2.4 Downside Risk Prediction and Threshold-Based Classification

Financial risk prediction is often framed as a binary classification problem, where the model predicts whether an adverse event will occur on the next trading day. This approach

is useful because it produces clear outputs, is less affected by short-term noise than regression-based forecasting and can be evaluated with metrics that are directly relevant to risk management (Peng and Yan, 2021). The threshold used to define a risk event is also important because it affects both how common the target class is and how economically meaningful the event is. In this study, a two percent daily price decline is used because it is a practical threshold for identifying significant downside movements in commodity markets.

Evaluating binary risk models requires more than accuracy, especially when risk events are rare. Recall is particularly important because missing a real risk event is usually worse than raising a false alarm. ROC-AUC measures how well the model separates risk from non-risk cases, while Brier Score and Log Loss assess whether the predicted probabilities are well calibrated. Using these metrics together provides a more complete comparison between the baseline and enhanced models than accuracy alone (Doshi-Velez and Kim, 2018).

2.5 Summary and Research Gaps

The literature shows that machine learning is now an established approach to financial risk prediction, with Random Forests offering a useful balance between predictive performance and interpretability. SHAP has become a leading method for explaining model outputs in finance and has already been applied to gold price forecasting and broader financial risk analysis. FinBERT-based sentiment analysis has also been shown to improve financial prediction by capturing information does not present in numerical market data, although results vary across asset classes and market conditions.

Despite this progress, three research gaps remain. First, explainable AI has been only lightly applied to downside risk prediction in coinage metal markets such as gold, silver, and copper, even though these markets are influenced by supply-side and geopolitical factors that differ from equity or credit markets. Second, few studies have combined FinBERT sentiment features and explainable machine learning in a single framework for metal market risk prediction. Third, many studies treat accuracy and interpretability as competing goals, whereas this project treats them as complementary by using SHAP to explain predictions and check whether they align with financial theory.

3 Research Methodology

3.1 Research Design

This project uses a quantitative, experimental design. It tests hypotheses about the value of sentiment analysis and the interpretability of machine learning outputs through

experiments on real financial data, with results measured against performance criteria. The study follows a positivist approach because it relies on observable, measurable phenomena and aims for findings that can be replicated and generalised (Doshi-Velez and Kim, 2018).

The methodology uses a comparative design with two models trained under the same conditions. The baseline model is a Random Forest classifier using technical price indicators and macroeconomic features only. The enhanced model uses the same features plus FinBERT sentiment scores extracted from financial news headlines. By keeping all other variables constant and changing only the sentiment input, the design isolates the effect of NLP-based sentiment analysis on predictive performance. This directly supports Objective 2 and Objective 4 and provides a clear basis for answering the research question.

3.2 Data Collection

3.2.1 Price and Macroeconomic Data

Historical OHLCV (Open, High, Low, Close, Volume) price data for gold (ticker: GC=F), silver (SI=F), and copper (HG=F) were collected covering the period from January 2020 to December 2025, providing approximately 1,500 trading days per metal. Data were retrieved using the yFinance Python library, which provides programmatic access to Yahoo Finance market data.

Four macroeconomic indicators were collected via yFinance: the US Dollar Index, VIX, 10-year Treasury yield, and S&P 500 daily return

3.2.2 Financial News Sentiment Data

Financial news headlines for gold, silver, and copper were collected using the NewsAPI service, which aggregates content from sources like Reuters, Bloomberg, the Financial Times, and MarketWatch. Queries used metal-specific terms such as "gold price," "silver market," and "copper commodity" to ensure the data remained relevant. I initially evaluated GDELT as an alternative source but replaced it with NewsAPI due to inconsistent data availability and quality.

Raw headline text was processed through the FinBERT model, which is a version of BERT pre-trained on financial texts (Araci, 2019; Liu et al., 2020). This model generated three probability scores for each headline: positive, negative, and neutral sentiment.

3.3 Data Preprocessing and Feature Engineering

3.3.1 Target Variable Construction

The binary target variable represents whether a downside risk event occurs on the next trading day. For each day t, the label is set to 1 if the return on day t is below negative two percent, and 0 otherwise. This forward-looking setup prevents look-ahead bias because the model uses only information available at day t to predict day t . The two percent threshold was chosen because it captures economically meaningful declines in commodity markets while still producing enough positive cases for training.

3.3.2 Technical Indicators

A comprehensive set of technical indicators was derived from OHLCV price data to capture momentum, trend, volatility, and volume. These were selected for their prominence in financial literature and their relevance to short-term price risk. The features engineered for each metal include:

- Simple Moving Averages (SMA): 5, 10, 20, and 50-day windows for multi-horizon trend analysis.
- Relative Strength Index (RSI): 14-day window to measure the speed and magnitude of price changes.
- Moving Average Convergence Divergence MACD: The difference between 12 and 26-day EMAs, including the signal line and histogram.
- Bollinger Bands: 20-day window including bandwidth and percentage B to measure volatility and relative price position.
- Average True Range (ATR) at a 14-day window, measuring market volatility by decomposing the entire range of an asset price for a given period.
- 20-day historical volatility, computed as the rolling standard deviation of log returns.
- Daily return, log return, high-low ratio, volume change, and 20-day volume moving average as additional price and volume features.

Finally, lag features at t = 1, t = 2, and t = 3 were computed for key variables to provide a short-term historical window and prevent look-ahead bias.

3.4 Model Development

3.4.1 Algorithm Selection

Random Forest was selected as the classification algorithm for both models due to four key factors. First, it effectively models the non-linear relationships common in financial data. Second, its ensemble structure makes it robust to the outliers and noise typical of financial time series. Third, it is directly compatible with SHAP TreeExplainer, which provides computationally efficient and theoretically exact feature explanations (Lundberg and Lee, 2017). Finally, Random Forests are well-established in financial risk research, providing a reliable benchmark for evaluating sentiment feature contributions (Peng and Yan, 2021).

3.4.2 Handling Class Imbalance

Since downside risk events (daily declines exceeding two percent) represent a minority class (8–12% of the data), class imbalance poses a significant modelling challenge. A naive model predicting "no risk" would achieve high overall accuracy while failing to detect actual risk events. To address this, the study uses scikit-learn's balanced class weight formula, which sets weights inversely proportional to class frequency. This approach penalises misclassifications of risk events more heavily than non-events, improving recall for the minority class without the need to discard training data.

3.4.3 Train, Validation and Test Split

The dataset for each metal was divided into training, validation, and test sets using a chronological 60/20/20 split. This means the first sixty percent of trading days form the training set, the next twenty percent form the validation set for tuning, and the final twenty percent form the test set for final evaluation. This split is strictly chronological and uses no random shuffling. This preserves the time order of the data and prevents look-ahead bias.

3.4.4 Hyperparameter Tuning

Hyperparameter optimization was performed using GridSearchCV with TimeSeriesSplit cross-validation, configured with five folds. The best hyperparameter combination was selected based on ROC-AUC score on the validation folds, as ROC-AUC is robust to class imbalance and directly measures the model's ability to discriminate between risk and non-risk days across all possible classification thresholds.

3.5 Explainability Framework

SHAP (SHapley Additive ExPlanations) was applied to both the baseline and enhanced models to generate interpretable explanations of their predictions. The TreeExplainer variant was used, which is specifically optimised for tree-based models and computes

exact Shapley values rather than approximations, ensuring that explanations are theoretically consistent and faithful to the model's actual decision-making process (Lundberg and Lee, 2017).

3.6 Evaluation Framework

Model performance was assessed using classification and probabilistic metrics to balance accurate event detection with reliable probability estimation. Probabilistic calibration was evaluated using the Brier Score, which measures the mean squared difference between predictions and outcomes, and Log Loss, which penalises overconfident errors. Well-calibrated probabilities are essential in risk management, where analysts must rank days by risk level and base decisions on the magnitude of the score rather than binary labels. Calibration curves were also generated to visually assess the alignment between predicted probabilities and observed event frequencies.

3.8 Limitations of Methodology

Several methodological limitations should be acknowledged. First, the NewsAPI free tier limits historical data access, resulting in sparse sentiment coverage for earlier periods (2020–2025). Days with missing headlines are assigned a neutral sentiment value (zero), which may underestimate the predictive contribution of sentiment if data availability is non-random (e.g., lower headline volume during low-volatility periods).

Second, financial markets are non-stationary, meaning feature-target relationships may shift over time. While walk-forward backtesting on 2025 data evaluates the model on recent information, it cannot guarantee future performance. Model outputs should therefore be treated as analytical indicators rather than definitive forecasts, and any production implementation would require periodic retraining to remain accurate.

4 Design and Implementation

4.1 System Architecture Overview

The system uses a six-stage data pipeline with each stage implemented as a standalone numbered Python script. This design was deliberately chosen to separate concerns clearly, making each component independently testable and maintainable. The pipeline progresses through six stages: data collection, feature engineering, sentiment scoring, model training, evaluation, and backtesting.

4.2 Database Design

The metal_risk_prediction PostgreSQL database was designed using a normalized relational schema to minimise data duplication and preserve referential integrity across the pipeline. The database comprises seven tables, each supporting a distinct stage of data storage and feature engineering.

Table 1: PostgreSQL database schema summary

Table	Primary Key	Description
metals	metal_id (INT)	Reference table storing the three metals: gold (1), silver (2), copper (3).
Price_data	price_id (SERIAL)	Daily OHLCV price data for each metal. UNIQUE constraint on (metal_id, date).
Technical_features	feature_id (SERIAL)	All engineered technical indicators per metal per day. NUMERIC(20,6) precision throughout to avoid overflow.
Macroeconomic_data	macro_id (SERIAL)	Daily values for DXY, VIX, 10-year Treasury yield, and S&P 500. Not metal-specific so shared across all three metals.
Sentiment_data	sentiment_id (SERIAL)	Individual headline sentiment scores from FinBERT, including positive, negative, and neutral probability scores per headline.
Daily_sentiment	daily_sent_id (SERIAL)	Daily aggregated sentiment features per metal: mean scores, headline count, positive/negative ratios, and sentiment standard deviation.

The use of NUMERIC(20,6) in the technical_features table was necessary to avoid overflow errors encountered during development when storing indicators such as volume-based measures, which vary substantially in scale.

4.2.1 Schema Structure and Table Design

The database uses six tables. The price_data table stores OHLCV information and includes CHECK constraints to ensure that all price and volume values are non-negative, which prevents invalid records from entering the processing stage.
The macroeconomic_data table is market-wide rather than metal-specific, so the same information is not stored three times. The sentiment_data and daily_sentiment tables store FinBERT-scored headlines and their daily aggregates. The two primary tables, metals and technical_features, are described below.

Metals

```
-- 1) METALS
CREATE TABLE IF NOT EXISTS metals (
  metal_id SERIAL PRIMARY KEY,
  symbol VARCHAR(10) UNIQUE NOT NULL,
  name VARCHAR(50) NOT NULL,
  -- ... (full schema: database/schema.sql)
);
```

The metals table stores the three metal identifiers. Other tables link to these via a metal_id to avoid repeating metal names across thousands of rows.
The market_type column separates precious metals from industrial metals because gold and silver react differently to risk than copper. The insert command uses a conflict rule so that data can be added safely without creating duplicate errors during development.

Technical features

```
-- 4) TECHNICAL FEATURES
CREATE TABLE IF NOT EXISTS technical_features (
  feature_id SERIAL PRIMARY KEY,
  metal_id   INTEGER NOT NULL REFERENCES metals(metal_id) ON DELETE CASCADE,
  date       DATE NOT NULL,
  daily_return NUMERIC(18, 10),
  log_return   NUMERIC(18, 10),
```

```
rsi_14 NUMERIC(6, 2),
-- ... MACD, Bollinger, SMA, EMA, volume columns
-- (full schema: database/schema.sql)
```

The technical_features table stores all calculated metrics. Each record is linked to a metal by its metal_id and identified by a date, with a constraint to prevent duplicate entries for the same day.

To save space and ensure accuracy, three levels of numerical precision are used:

- RSI values use NUMERIC(6,2) because they are always between 0 and 100.
- Return and ratio columns use NUMERIC(18,10) because they require high precision for small decimal values.
- Moving averages and price columns use NUMERIC(20,6) because they store prices that can be large, such as gold trading above 2,000 USD.

The table also uses a delete constraint so that if a metal is removed from the reference list, all related feature records are automatically cleaned up.

4.2.2 Precision and Data Integrity Decisions

During development a numeric overflow error was encountered when inserting volume-based indicators into the technical_features table. The error was reproduced and diagnosed using pgAdmin as shown in Figure below.

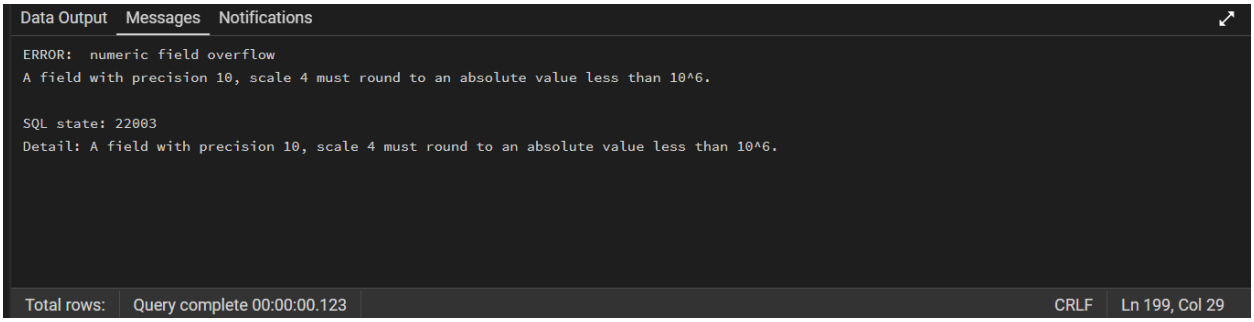


Figure 1: Numeric field overflow error due to insufficient column precision.

The error confirms that NUMERIC(10,4) supports a maximum absolute value less than 10^6, meaning any volume-based indicator exceeding 999,999 would cause the entire

batch insertion to fail. Gold futures volume values regularly exceed this range during high-activity trading sessions, causing the pipeline to crash silently on those dates.

4.2.3 Idempotent Insertion Strategy

All six data insertion scripts were designed to be able to re-run multiple times without creating duplicate records. The insertion function for price data is shown below in 01_data_collection.

```
def insert_price_data(conn, metal_id, df):
  records = [(metal_id, r["date"], float(r["open"]), ...)
        for _, r in df.iterrows()]
  sql = """INSERT INTO price_data (...) VALUES %s
       ON CONFLICT (metal_id, date) DO NOTHING;"""
  # ... (full implementation: scripts/01_data_collection.py)
  execute_values(cur, sql, records, page_size=2000)
```

The ON CONFLICT DO NOTHING clause ensures the script can be re-run safely at any point without requiring the database to be manually cleared and execute_values sends the entire batch in a single database call for efficiency.

4.2.4 Sentiment Feature Aggregation

The sentiment pipeline stores individual FinBERT-scored headlines in the sentiment_data table, but the Random Forest model requires a single set of daily features per metal rather than a variable number of individual headline scores. The aggregation from raw headlines to daily model-ready features is performed entirely in SQL, as shown below in 03_sentiment_pipeline.

```
INSERT INTO daily_sentiment (metal_id, date, avg_sentiment,
  avg_positive, avg_negative, avg_neutral, headline_count,
  positive_ratio, negative_ratio, sentiment_std)
SELECT metal_id, date,
  AVG(sentiment_score), AVG(positive_score),
  -- ... (full query: scripts/03_sentiment_pipeline.py)
```

The aggregation produces eight daily features from the raw headline scores. AVG(sentiment_score) computes the mean sentiment across all headlines published on a given day, producing a value between -1 and +1. The positive_ratio and negative_ratio columns use a CASE statement to convert categorical sentiment labels into proportions.

For example a positive_ratio of 0.7 means 70% of that day's headlines were classified as positive by FinBERT.

4.3 Data Collection Pipeline

4.3.1 Script 01: Price and Macroeconomic Data Collection

Script 01 collects historical OHLCV price data and macroeconomic indicators via the yFinance API and stores them in the PostgreSQL database using idempotent batch insertions.

The script consists of seven functions:

- get_metal_map: retrieves metal identifiers from the database
- _flatten_yfinance_columns: handles yFinance MultiIndex column inconsistency
- download_prices: downloads and cleans OHLCV price data
- download_macro: downloads four macroeconomic indicators
- insert_price_data: inserts price data using batch insertion
- verify_counts: post-insertion verification query
- main: orchestrates the full pipeline

The two most technically significant functions are download_prices and download_macro, documented in full below. These were selected because both required non-trivial handling of yFinance version inconsistencies and data quality issues that are not immediately obvious from the function signatures alone.

download_prices

Requirements

The download_prices function must retrieve historical daily OHLCV price data for a given metal ticker from Yahoo Finance covering January 2020 to December 2025. It must handle inconsistencies in yFinance column output across library versions and return a clean DataFrame ready for database insertion.

Design

```
FUNCTION download_prices(ticker, start, end):
  Download OHLCV data from yFinance
  IF empty: RETURN None

  Flatten MultiIndex columns
```

```
  Standardise date and column names to lowercase
  Validate all required columns exist
  Remove rows with missing price values
  RETURN cleaned DataFrame
END FUNCTION
```

Implementation

```
def download_prices(ticker, start="2020-01-01", end="2025-12-31"):
  df = yf.download(ticker, start=start, end=end,
            progress=False, auto_adjust=False)
  if df is None or df.empty:
    return None
  df = _flatten_yfinance_columns(df)
  df = df.reset_index()
  # ... column standardisation and validation (see scripts/01_data_collection.py)
  df["date"] = pd.to_datetime(df["date"]).dt.date
  return df[["date","open","high","low","close","volume","adjusted_close"]]
```

Testing

Insertion of 1,509 rows for gold, 1,509 for silver, and 1,510 for copper confirms clean DataFrames were returned across the full 2020-2025 window.

```
Postgres password:
✓ Connected to DB: metal_risk_prediction

--- GOLD (GC=F) ---
✓ Insert attempted: 1509 rows (duplicates ignored)

--- SILVER (SI=F) ---
✓ Insert attempted: 1509 rows (duplicates ignored)

--- COPPER (HG=F) ---
✓ Insert attempted: 1510 rows (duplicates ignored)
```

Figure 2: Successful insertion of price data for gold, silver, and copper.

```
Metal coverage:
  Copper: 1510 rows | 2020-01-02 -> 2025-12-30
  Gold: 1509 rows | 2020-01-02 -> 2025-12-30
  Silver: 1509 rows | 2020-01-02 -> 2025-12-30
```

Figure 3: Confirmed date range coverage for all metals.

download_macro

Requirements

The download_macro function must download daily values for four macroeconomic indicators and merge them into a single DataFrame aligned by date. Missing values must be handled using forward-fill imputation and S&P 500 daily return computed as a derived feature.

Design

```
FUNCTION download_macro(start, end):
  Define four tickers: DXY, VIX, TNX, S&P500

  FOR each ticker:
    Download closing price from yFinance
    Store as named column (usd_index, vix, etc.)

  Merge all four DataFrames on date using outer join
  Sort by date
  Forward-fill any missing values
  Compute sp500_return as daily percentage change
  Remove rows where any core indicator is missing
  RETURN cleaned DataFrame
END FUNCTION
```

Implementation

```
def download_macro(start="2020-01-01", end="2025-12-31"):
  tickers = {"DX-Y.NYB": "usd_index", "^VIX": "vix",
        "^TNX": "treasury_yield_10y", "^GSPC": "sp500_close"}
  frames = []
  for tkr, col in tickers.items():
    df = yf.download(tkr, start=start, end=end, progress=False)
    # … column flattening and renaming
    frames.append(df[["date", col]])
  macro = frames[0]
  for f in frames[1:]:
    macro = macro.merge(f, on="date", how="outer")
  macro = macro.sort_values("date").ffill()
  macro["sp500_return"] = macro["sp500_close"].pct_change()
```

```
    return macro.dropna(subset=["usd_index","vix","treasury_yield_10y",
                                "sp500_close","sp500_return"])
```

Testing

Insertion of 1,508 macroeconomic rows confirms all four indicators were downloaded, merged, and cleaned correctly. The lower row count reflects dropna removing days where any macro indicator was unavailable.

```
--- MACRO (DXY, VIX, TNX, S&P500) ---
✓ Insert attempted: 1508 rows (duplicates ignored)
```

Figure 4: Successful insertion of macroeconomic data.

4.3.2 Script 02: Feature Engineering

Script 02 reads raw OHLCV data from the database, engineers all technical indicators required as model input features, and stores the computed features back into the technical_features table.

- create_db_connection: establishes the SQLAlchemy database connection
- calculate_rsi: computes the Relative Strength Index
- calculate_macd: computes the Moving Average Convergence Divergence
- calculate_bollinger: computes Bollinger Bands and bandwidth
- load_price_data: loads raw OHLCV data from the database for one metal
- build_features: engineers all technical indicators from price data
- build_risk_events: constructs the binary risk event labels
- upsert_technical_features: inserts engineered features into the database
- main: orchestrates the full feature engineering pipeline

calculate_rsi and build_features are documented in full below, as they form the core of the feature engineering logic and contain the most technically significant design decisions in the script.

Calculate_rsi

Requirements

The calculate_rsi function must compute the Relative Strength Index using Wilder's smoothing method over a 14-day period, returning values bounded between 0 and 100 that are consistent and comparable across all three metals.

Design

```
FUNCTION calculate_rsi(series, period):
  Compute daily price changes
  Separate changes into gains and losses
  Compute rolling mean of gains over period
  Compute rolling mean of losses over period
  Compute RS = average gain / average loss
  RETURN 100 - (100 / (1 + RS))
END FUNCTION
```

Implementation

```
def calculate_rsi(series: pd.Series, period=14):
  delta = series.diff()
  gain = delta.where(delta > 0, 0).rolling(period).mean()
  loss = (-delta.where(delta < 0, 0)).rolling(period).mean()
  rs = gain / loss
  return 100 - (100 / (1 + rs))
```

Testing

Insertion of valid feature rows for all three metals with no overflow errors confirms RSI values were correctly bounded between 0 and 100, matching the NUMERIC(6,2) constraint.

```
✓ Connected to database: metal_risk_prediction

--- Gold (metal_id=1) ---
✓ Inserted technical_features: 1452
✓ Inserted risk_events: 1508

--- Silver (metal_id=2) ---
✓ Inserted technical_features: 1389
✓ Inserted risk_events: 1508

--- Copper (metal_id=3) ---
✓ Inserted technical_features: 1455
✓ Inserted risk_events: 1509

======================================================================
DONE
======================================================================
Total technical_features inserted: 4296
Total risk_events inserted: 4525
```

Figure 5: Successful completion of feature engineering and risk event labeling.

Built_features

Requirements

The build_features function must compute all technical indicators required as model input features from the raw OHLCV data. It must handle infinite values produced by ratio calculations before returning the DataFrame to prevent database insertion failures.

Design

```
FUNCTION build_features(df):
  Compute daily return and log return from close price

  Compute SMAs at 5, 10, 20, 50 day windows
  Compute EMAs at 12 and 26 day windows

  Compute Bollinger Bands (upper, middle, lower, width)
  Compute RSI at 14 day window
  Compute MACD, signal line and histogram

  Compute high-low range and high-low ratio
  Compute volume change and 20 day volume moving average

  Replace all inf and -inf values with NaN
  RETURN DataFrame with all features
END FUNCTION
```

Implementation

```
def build_features(df: pd.DataFrame):
  df = df.copy()
  df["daily_return"] = df["close"].pct_change()
  df["log_return"]   = np.log(df["close"] / df["close"].shift(1))
  df["sma_5"]      = df["close"].rolling(5).mean()
  # ... sma_10, sma_20, sma_50, ema_12, ema_26, Bollinger,
  # ... RSI, MACD, high-low features, volume features
  # (full implementation: scripts/02_feature_engineering.py)
  df.replace([np.inf, -np.inf], np.nan, inplace=True)
  return df
```

Testing

Insertion of 4,296 technical feature rows across all three metals confirms all indicators were produced correctly with no overflow errors. The inf/-inf replacement was tested on the volume_change column, confirming it prevents PostgreSQL rejection where previous-day volume was zero.

4.3.3 Script 03: FinBERT Sentiment Pipeline

Script 03 collects financial news headlines from NewsAPI, scores each headline using the FinBERT sentiment model, aggregates the scores to daily level, and stores the results in the PostgreSQL database.

- create_db_connection: establishes the SQLAlchemy database connection
- FinBERTAnalyzer: loads and runs the FinBERT model for sentiment scoring
- collect_newsapi_headlines: fetches headlines from NewsAPI for a given keyword
- collect_headlines_for_metal: collects and deduplicates headlines for all keywords of a metal
- insert_sentiment_data: inserts individual scored headlines into the database
- aggregate_daily_sentiment: aggregates headline scores to daily features
- main: orchestrates the full sentiment pipeline

FinBERTAnalyzer is documented in full below as it is the most technically complex component, responsible for loading and running the transformer model in batched inference mode.

FinBERTAnalyzer

Requirements

The FinBERTAnalyzer class must load the ProsusAI/finbert model once at startup and process headlines in batches of 32, producing positive, negative, and neutral probability scores and a composite sentiment score on the interval -1 to +1.

Design

```
CLASS FinBERTAnalyzer:
  INIT:
    Load FinBERT tokenizer and model from HuggingFace
    Set model to evaluation mode

  FUNCTION analyze(headlines):
    FOR each batch of 32 headlines:
      Tokenize batch with padding and truncation
```

```
    Run model forward pass with no gradient computation
    Apply softmax to get probabilities
    FOR each headline in batch:
      Extract positive, negative, neutral probabilities
      Compute sentiment score = positive - negative
      Store result
  RETURN all results
END CLASS
```

Implementation

```
class FinBERTAnalyzer:
  def __init__(self):
    self.tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert")
    self.model     = AutoModelForSequenceClassification.from_pretrained(
                "ProsusAI/finbert")
    self.model.eval()

  def analyze(self, headlines: list) -> list:
    results = []
    for i in range(0, len(headlines), 32):
      batch  = headlines[i:i+32]
      inputs = self.tokenizer(batch, padding=True, truncation=True,
                  max_length=128, return_tensors="pt")
      with torch.no_grad():
        probs = torch.nn.functional.softmax(
          self.model(**inputs).logits, dim=-1)
      # ... extract pos/neg/neu scores and build results list
      # (full implementation: scripts/03_sentiment_pipeline.py)
    return results
```

Testing

The model loaded successfully and scored all 1,321 headlines, which consisted of 346 negative, 692 neutral, and 283 positives. The dominance of neutral classifications is consistent with expected financial news behaviour.

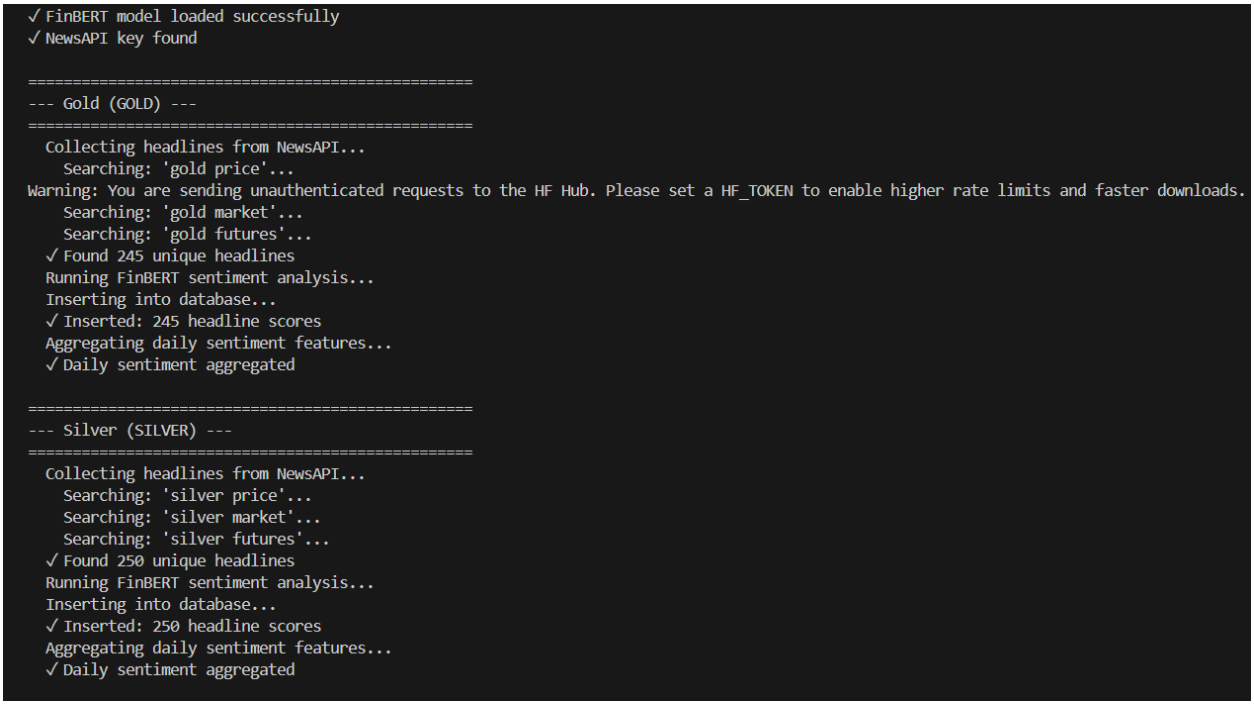


Figure 6: Successful sentiment scoring of gold and silver headlines.

```
==================================================
--- Copper (COPPER) ---
==================================================
  Collecting headlines from NewsAPI...
    Searching: 'copper price'...
    Searching: 'copper market'...
    Searching: 'copper futures'...
  ✓ Found 202 unique headlines
  Running FinBERT sentiment analysis...
  Inserting into database...
  ✓ Inserted: 202 headline scores
  Aggregating daily sentiment features...
  ✓ Daily sentiment aggregated

======================================================================
VERIFICATION
======================================================================
  Total headlines scored:    1321
  Daily sentiment records:   168
  Copper: 410 headlines | 56 days | 2026-01-27 to 2026-03-24
  Gold: 446 headlines | 56 days | 2026-01-27 to 2026-03-24
  Silver: 465 headlines | 56 days | 2026-01-27 to 2026-03-24

  Sentiment distribution:
    negative: 346
    neutral: 692
    positive: 283

✓ SENTIMENT PIPELINE COMPLETE
```

Figure 7: Successful sentiment scoring and distribution for copper.

4.4 Model Training Implementation

4.4.1 Script 04: Model Training

Script 04 loads all feature data from the database, constructs the binary target variable, splits the data chronologically, trains both baseline and enhanced Random Forest models, generates SHAP explanations, and saves all trained models and outputs to disk.

- get_engine: establishes the SQLAlchemy database connection
- load_feature_matrix: loads and merges technical, macroeconomic, and sentiment features from the database
- build_target_and_lag_features: constructs the binary target variable and lag features
- get_feature_sets: defines baseline and enhanced feature column lists

- temporal_split: divides the dataset chronologically into training, validation, and test sets
- train_random_forest: trains the Random Forest classifier with hyperparameter tuning
- evaluate_model: computes evaluation metrics on a given dataset split
- generate_shap_explanations: applies SHAP TreeExplainer to generate feature attributions
- main: orchestrates the full training pipeline for all three metals

build_target_and_lag_features, temporal_split, and train_random_forest are documented in full below as they contain the core modelling decisions.

Build_target_and_lag_features

Requirements

The build_target_and_lag_features function must construct the binary target variable using a forward shift to prevent look-ahead bias, and engineer lag features at t-1, t-2, and t-3 for six key variables to provide the model with short-term temporal context.

Design

```
RISK_THRESHOLD = -0.02

def build_target_and_lag_features(df: pd.DataFrame) -> pd.DataFrame:
  df = df.copy()
  df['target'] = (df['daily_return'].shift(-1) < RISK_THRESHOLD).astype(int)
  # ... lag features at t-1, t-2, t-3 for six key columns
  # (full implementation:   scripts/04_model_training.py)
  return df
```

Testing

Gold produced 49 risk events from 1,450 samples (3.4%), silver 147 from 1,386 (10.6%), and copper 112 from 1,452 (7.7%). The last row target value of 0 for all metals confirms shift(-1) correctly drops the boundary row, preventing look-ahead bias.

```
Target distribution:
target
0    1401
1      49
Name: count, dtype: int64
Target rate: 0.034
First row target: 1
Last row target: 0
```

Figure 8: Gold target variable construction and risk event summary.

```
Target distribution:
target
0    1239
1     147
Name: count, dtype: int64
Target rate: 0.106
First row target: 0
Last row target: 0
```

Figure 9: Silver target variable construction and risk event summary.

```
Target distribution:
target
0    1340
1     112
Name: count, dtype: int64
Target rate: 0.077
First row target: 0
Last row target: 0
```

Figure 10: Copper target variable construction and risk event summary.

Temporal_split

Requirements

The temporal_split function must divide the dataset into training, validation, and test sets using a strictly chronological 60/20/20 split with no random shuffling, preserving temporal order to prevent look-ahead bias.

Design

FUNCTION temporal_split(df):
  Calculate total number of rows
  Set train_end = 60% of total rows
  Set val_end = 80% of total rows

```
  train = rows from start to train_end
  val = rows from train_end to val_end
  test = rows from val_end to end

  RETURN train, val, test
END FUNCTION
```

Implementation

```
TRAIN_RATIO   = 0.60
VAL_RATIO     = 0.20
..
def temporal_split(df: pd.DataFrame):
..
n = len(df)
  train_end = int(n * TRAIN_RATIO)
  val_end   = int(n * (TRAIN_RATIO + VAL_RATIO))

  train = df.iloc[:train_end].copy()
  val   = df.iloc[train_end:val_end].copy()
  test  = df.iloc[val_end:].copy()
..
  return train, val, test
-
```

Testing

No overlap exists between any of the three sets and chronological ordering is preserved throughout.

```
Split sizes → Train: 870 | Val: 290 | Test: 290
Train period: 2020-03-17 → 2023-09-01
Val period  : 2023-09-05   → 2024-10-28
Test period : 2024-10-29  → 2025-12-30
```

Figure 11: Gold Split

```
Split sizes → Train: 831 | Val: 277 | Test: 278
Train period: 2020-03-18 → 2023-09-18
Val period  : 2023-09-19   → 2024-11-12
Test period : 2024-11-13  → 2025-12-30
```

Figure 12: Silver Split

```
Split sizes → Train: 871 | Val: 290 | Test: 291
Train period: 2020-03-18 → 2023-09-01
Val period  : 2023-09-05   → 2024-10-25
Test period : 2024-10-28  → 2025-12-30
```

Figure 13: Copper Split

Train_random_forest

Requirements

The train_random_forest function must train a Random Forest classifier using balanced class weighting and GridSearchCV with TimeSeriesSplit cross-validation, returning the best estimator selected based on ROC-AUC performance on the validation folds.

Design

FUNCTION train_random_forest(X_train, y_train, label):
  Compute balanced class weights from training label distribution

  Define parameter grid:
    n_estimators: [100, 200, 300]
    max_depth: [5, 10, 15, None]
    min_samples_split: [2, 5, 10]
    min_samples_leaf: [1, 2, 4]
    max_features: [sqrt, log2]

  Create base RandomForestClassifier with class weights
  Create TimeSeriesSplit with 5 folds
  Run GridSearchCV scoring on ROC-AUC

  RETURN best estimator
END FUNCTION

Implementation

def train_random_forest(X_train, y_train, label):

```
weights = compute_class_weight('balanced',
                    classes=np.unique(y_train), y=y_train)
param_grid = {'n_estimators': [100,200,300],
        'max_depth': [5,10,15,None],
        'min_samples_split': [2,5,10],
        'min_samples_leaf': [1,2,4],
        'max_features': ['sqrt','log2']}
# ... GridSearchCV with TimeSeriesSplit(n_splits=5), scoring='roc_auc'
# (full implementation: scripts/04_model_training.py)
return grid_search.best_estimator_
```

Testing

Class weights confirm correct minority class upweighting of gold 16.7x, silver 4.4x, copper 6.1x. Different optimal configurations per metal confirm each asset has a distinct feature-outcome relationship.

```
    Class weights: {0: np.float64(0.5154028436018957), 1: np.float64(16.73076923076923)}
Fitting 5 folds for each of 216 candidates, totalling 1080 fits

    Best CV ROC-AUC : 0.7872
    Best params     : {'max_depth': 5, 'max_features': 'log2', 'min_samples_leaf': 4, 'min_samples_split': 2, 'n_estimators': 100}
```

Figure 14: Optimized gold model parameters and class weights.

```
    Class weights: {0: np.float64(0.5637720488466758), 1: np.float64(4.420212765957447)}
Fitting 5 folds for each of 216 candidates, totalling 1080 fits

    Best CV ROC-AUC : 0.5531
    Best params     : {'max_depth': None, 'max_features': 'log2', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
```

Figure 15: Optimized silver model parameters and class weights.

```
    Class weights: {0: np.float64(0.544375), 1: np.float64(6.133802816901408)}
Fitting 5 folds for each of 216 candidates, totalling 1080 fits

    Best CV ROC-AUC : 0.5515
    Best params     : {'max_depth': 5, 'max_features': 'log2', 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 200}
```

Figure 16: Optimized copper model parameters and class weights.

4.4.2 Script 05: Model Evaluation

This part of the code loads the trained models to generate the results discussed in the analysis. It recreates the exact feature matrix and test split used during training to ensure that performance is measured on unseen data. It calculates all performance metrics for both the baseline and enhanced models for each metal, creates all visual charts, and saves a summary file of the results.

- plot_roc_curve: ROC curves for baseline and enhanced models
- plot_pr_curve: precision-recall curves

- plot_calibration: probability calibration plots
- plot_confusion: confusion matrices at the 0.5 threshold
- main: evaluation for all three metals and saves all outputs

The script organizes these steps to evaluate each metal in turn and save the final outputs. The process focuses on two key tasks: computing the performance metrics and generating calibration curves to verify the accuracy of the risk scores.

The first block shows how all six-evaluation metrics are computed inline for each model variant.

-

```
for model, y_prob, y_pred, label in [
  (model_b, y_prob_b, y_pred_b, 'Baseline'),
  (model_e, y_prob_e, y_pred_e, 'Enhanced')
]:
  all_metrics.append({
    'metal':    metal_name,
    'model':    label,
    'roc_auc':  roc_auc_score(y_test, y_prob),
    'avg_prec': average_precision_score(y_test, y_prob),
    'brier':    brier_score_loss(y_test, y_prob),
    'log_loss': log_loss(y_test, y_prob),
    'f1':       f1_score(y_test, y_pred, zero_division=0),
    'precision': precision_score(y_test, y_pred, zero_division=0),
    'recall':   recall_score(y_test, y_pred, zero_division=0),
  })
```

-

ROC-AUC and Average Precision are calculated using the raw probability scores instead of binary "yes or no" predictions. This allows us to measure how well the model separates risk from non-risk regardless of the chosen threshold. For F1-score, precision, and recall, the code is set to handle cases where the model predicts only "no risk." Because risk events occur only 7.3% of the time, the model often stays below the 0.5 threshold. The code handles this by treating zero-division as an expected outcome rather than an error.

The second block shows the calibration curve implementation with the quantile binning strategy.

-

```
frac_b, mean_b = calibration_curve(
  y_true, y_prob_base,
```

```
    n_bins=10,
    strategy='quantile'
)
```

Testing

Script 05 was validated by running it end to end and confirming all outputs were generated successfully. The summary output confirms all six model variants were evaluated correctly.

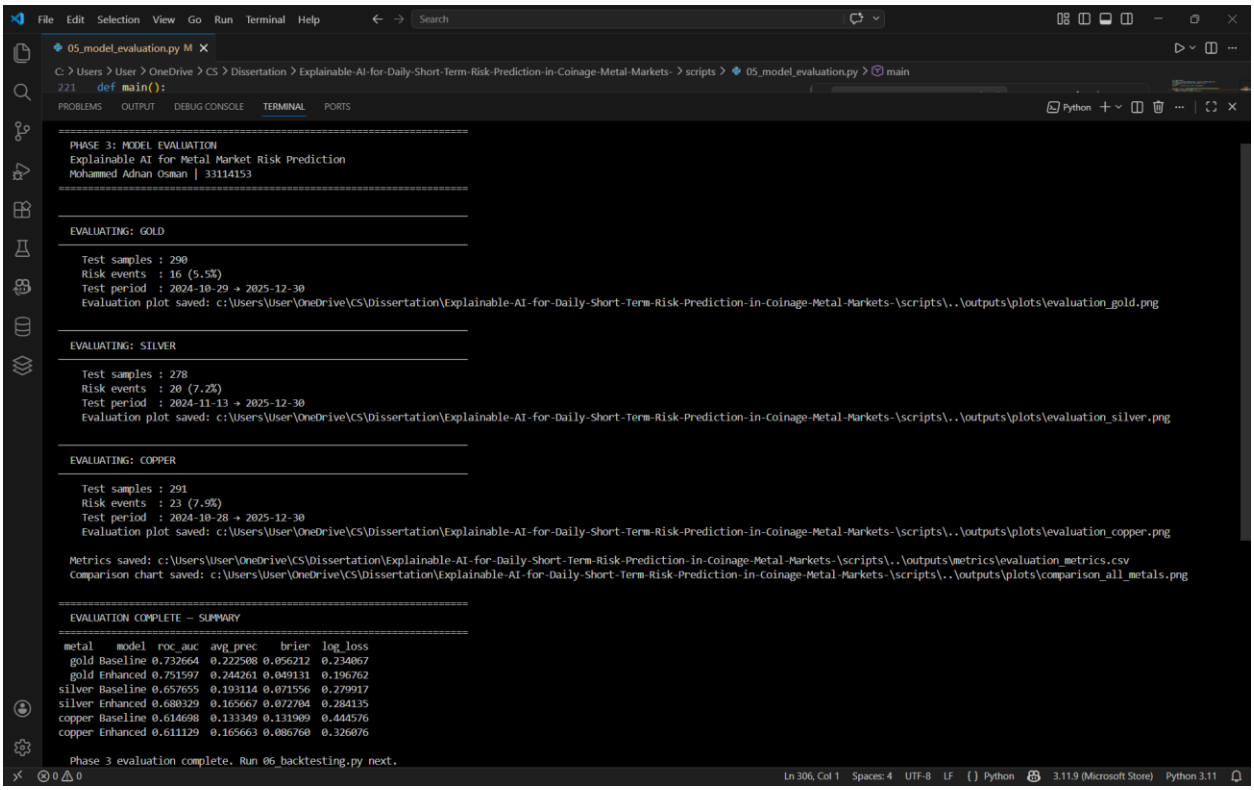


Figure 17: Final model evaluation metrics and performance summary.

4.4.3 Script 06: Back testing

Script 06 loads the trained models and runs walk-forward backtesting on unseen 2025 data, producing daily risk probability scores, timeline charts, SHAP waterfall plots, and prediction CSV files for each metal.

- get_engine: establishes the SQLAlchemy database connection
- load_full_dataset: loads and reconstructs the full feature matrix from the database
- walk_forward_predict: generates sequential daily risk predictions on the 2025 window
- plot_risk_timeline: plots daily risk probability over time with actual risk events marked

walk_forward_predict is documented in full below as it is the core backtesting function, responsible for generating sequential daily predictions on unseen 2025 data.

walk_forward_predict

Requirements

The walk_forward_predict function must generate daily risk probability scores for each trading day in the 2025 backtest window using only features available on that day, returning a DataFrame containing the date, actual risk label, predicted probability, and binary prediction.

Design

```
FUNCTION walk_forward_predict(model, df_full, features, start_date, end_date):
  Filter df_full to backtest window
  IF empty: return empty DataFrame with warning

  Extract feature matrix for backtest window
  Generate risk probabilities using predict_proba
  Generate binary predictions using predict

  Construct results DataFrame with:
    date, daily_return, actual_risk,
    risk_prob, predicted, correct

  RETURN results DataFrame
END FUNCTION
```

Implementation

```
def walk_forward_predict(model, df_full, features,
                         start_date, end_date):
  backtest_df = df_full[
    (df_full['date'] >= start_date) &
    (df_full['date'] <= end_date)
  ].copy()
  if len(backtest_df) == 0:
    return pd.DataFrame()
  # ... predict_proba and predict, return results DataFrame
  # (full implementation: scripts/06_backtesting.py)
```

Testing

The gold enhanced model achieved a backtest ROC-AUC of 0.847, substantially higher than the 0.752 test set score, demonstrating strong generalisation to 2025 market conditions.

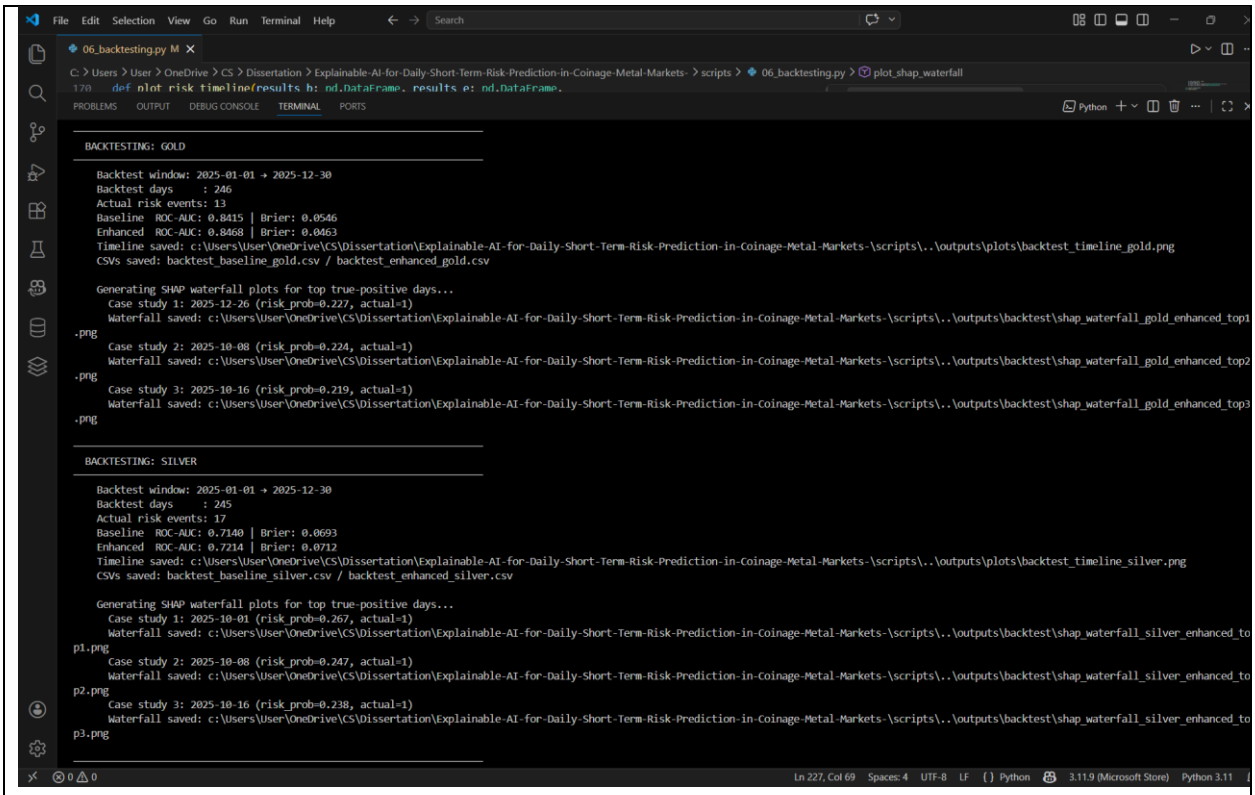


Figure 18: Walk-forward back testing results for gold and silver.

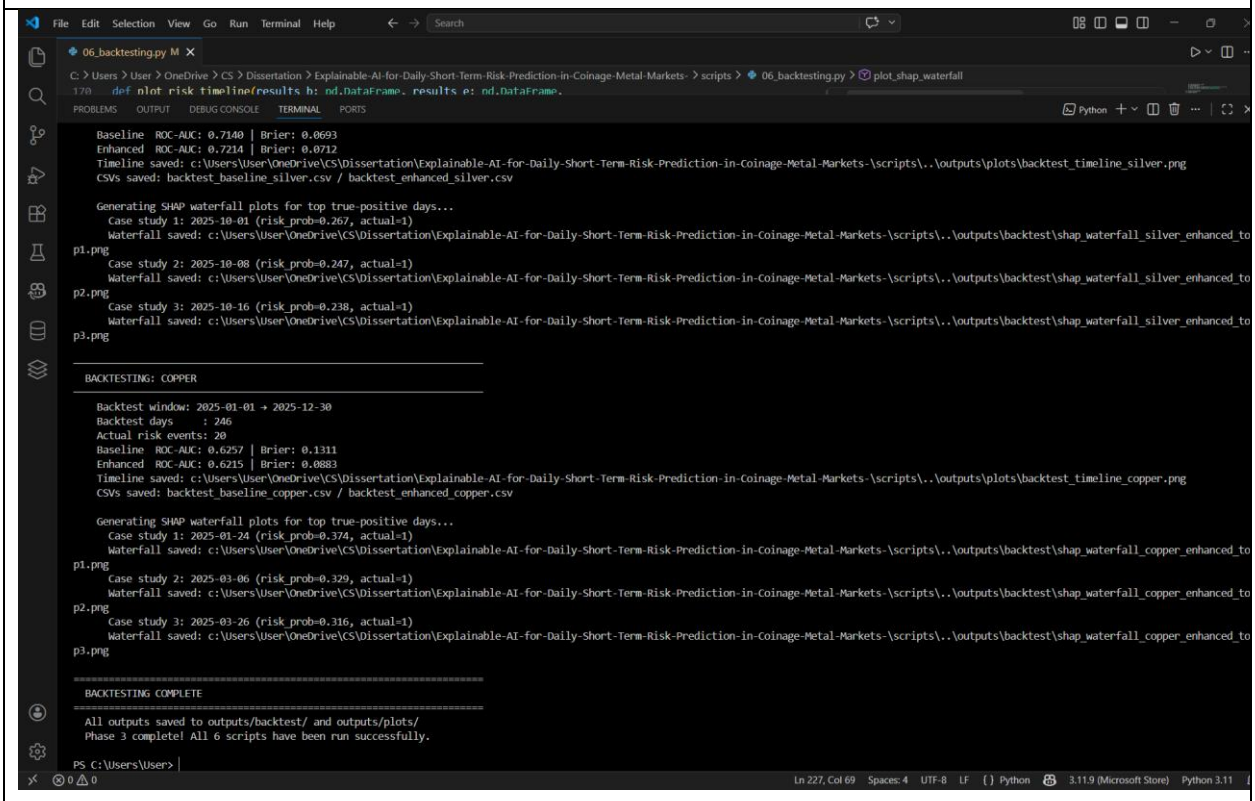


Figure 19: Walk-forward back testing results and completion status.

4.4.4 Script 07: SHAP Regeneration

Script 07 reloads the trained models and regenerates all SHAP bar charts and beeswarm plots using a corrected SHAP value extraction approach that handles array shape inconsistencies across SHAP library versions.

- get_engine: The SQLAlchemy database connection
- load_test_data: loads and reconstructs the test split from the database
- get_shap_values: extracts class-1 SHAP values handling all possible output shapes
- make_shap_plots: SHAP bar charts and beeswarm summary plots
- main: SHAP regeneration for all three metals and both model variants

get_shap_values is documented in full below as it contains the version-handling logic that resolved a critical SHAP API inconsistency encountered during development.

Get_shap_values

Requirements

The get_shap_values function must extract class-1 SHAP values from the raw TreeExplainer output regardless of array shape, which varies between SHAP library versions, returning a consistent two-dimensional array of shape (n_samples, n_features).

Design

```
FUNCTION get_shap_values(model, X_test):
  Create TreeExplainer with tree_path_dependent perturbation
  Compute raw SHAP values
  Convert to numpy array

  IF shape is (2, n_samples, n_features):
    RETURN arr[1]  #second element is positive class
  IF shape is (n_samples, n_features, 2):
    RETURN arr[:, :, 1]  #last dimension index 1 is positive class
  IF shape is (n_samples, n_features):
    RETURN arr  #already single output
  ELSE:
    RETURN arr  #fallback
END FUNCTION
```

Implementation

```
def get_shap_values(model, X_test):
  explainer = shap.TreeExplainer(
    model,
    feature_perturbation='tree_path_dependent'
  )
  raw = explainer.shap_values(X_test)
  arr = np.array(raw)

  if arr.ndim == 3 and arr.shape[0] == 2:
    return arr[1]

  if arr.ndim == 3 and arr.shape[2] == 2:
    return arr[:, :, 1]

  if arr.ndim == 2:
    return arr

  return arr
```

Testing

The three-branch conditional ensures correct positive class values are extracted regardless of which SHAP version is installed, making the pipeline robust across different deployment environments.

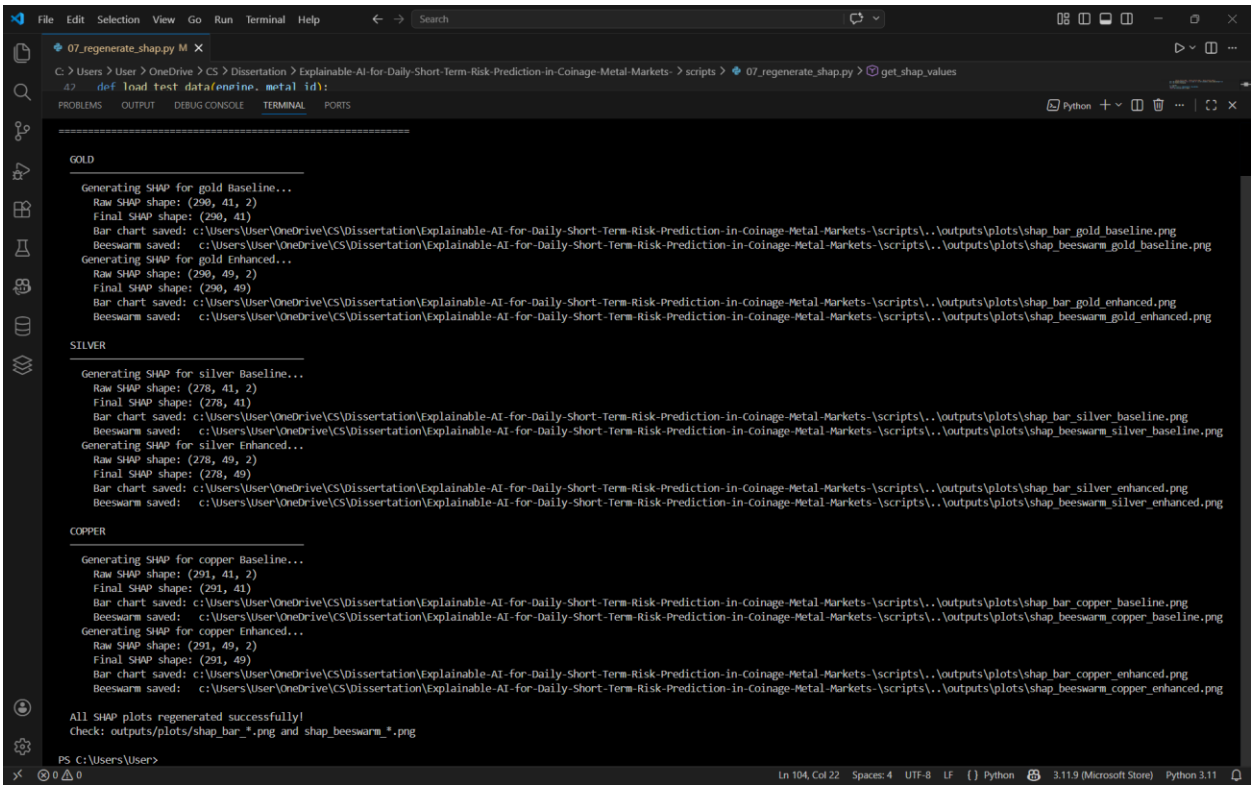


Figure 20: SHAP plot regeneration for all models.

4.5 Stream-lit Dashboard

The system includes an interactive dashboard built with Streamlit, providing an interface to explore model outputs, SHAP explanations, and live risk predictions. This dashboard meets the real-time display requirement and serves as the primary demonstration tool for the oral presentation.

The dashboard has four pages. The Live Prediction page fetches market data via yFinance, engineers 49 features, and runs the enhanced model to display colour-coded risk probabilities and real-time SHAP charts. The Model Performance page compares test-set metrics for the baseline and enhanced models, while the SHAP Analysis page presents the bar charts and plots produced in Script 07. The Back testing page shows ROC-AUC scores and timeline plots from Script 06.

Testing and Verification

On 27 April 2026, the dashboard was tested using live market data. The Live Prediction page generated risk scores for gold, silver, and copper, with all three classified as low risk. The SHAP charts confirmed that treasury_yield_10y was the main driver for gold, matching the global analysis in Chapter 5.

The Model Performance page also correctly displayed gold metrics including ROC-AUC 0.7516, Brier Score 0.0491, and Log Loss 0.1968. These values are consistent with the results from Script 05.

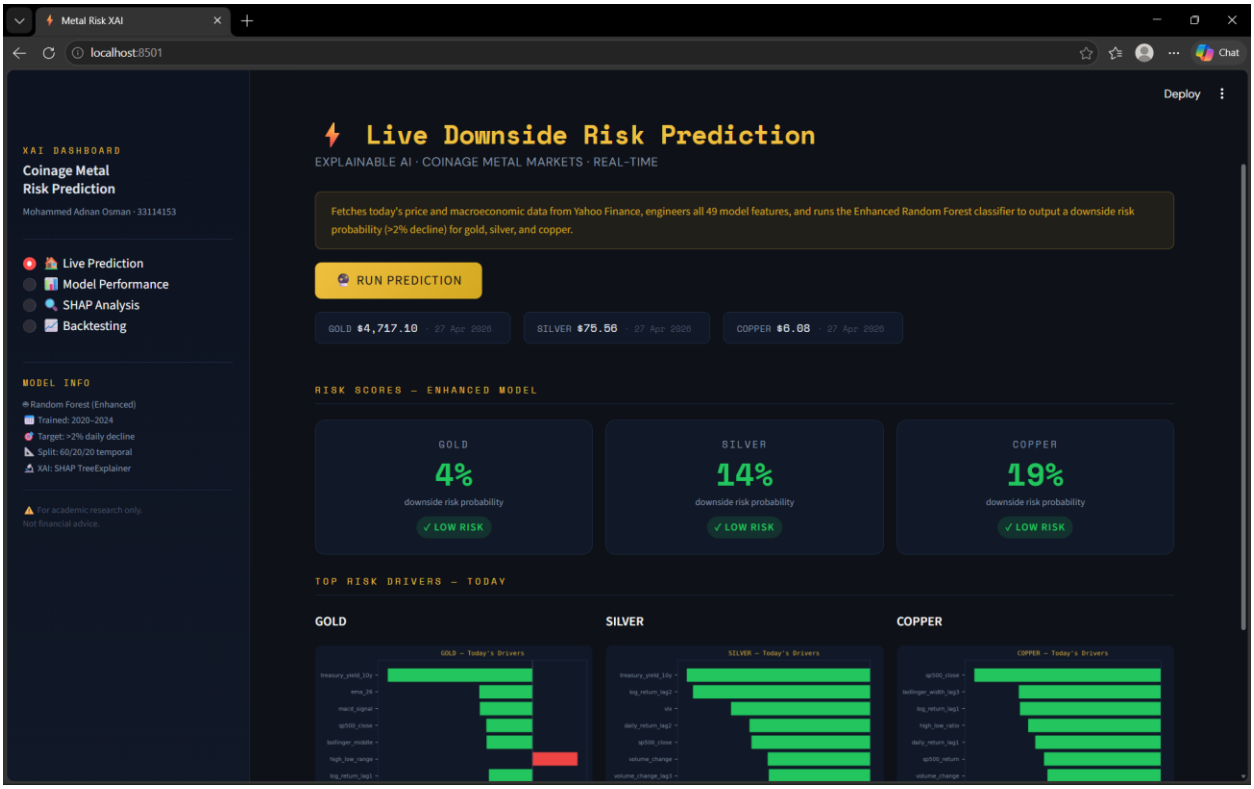
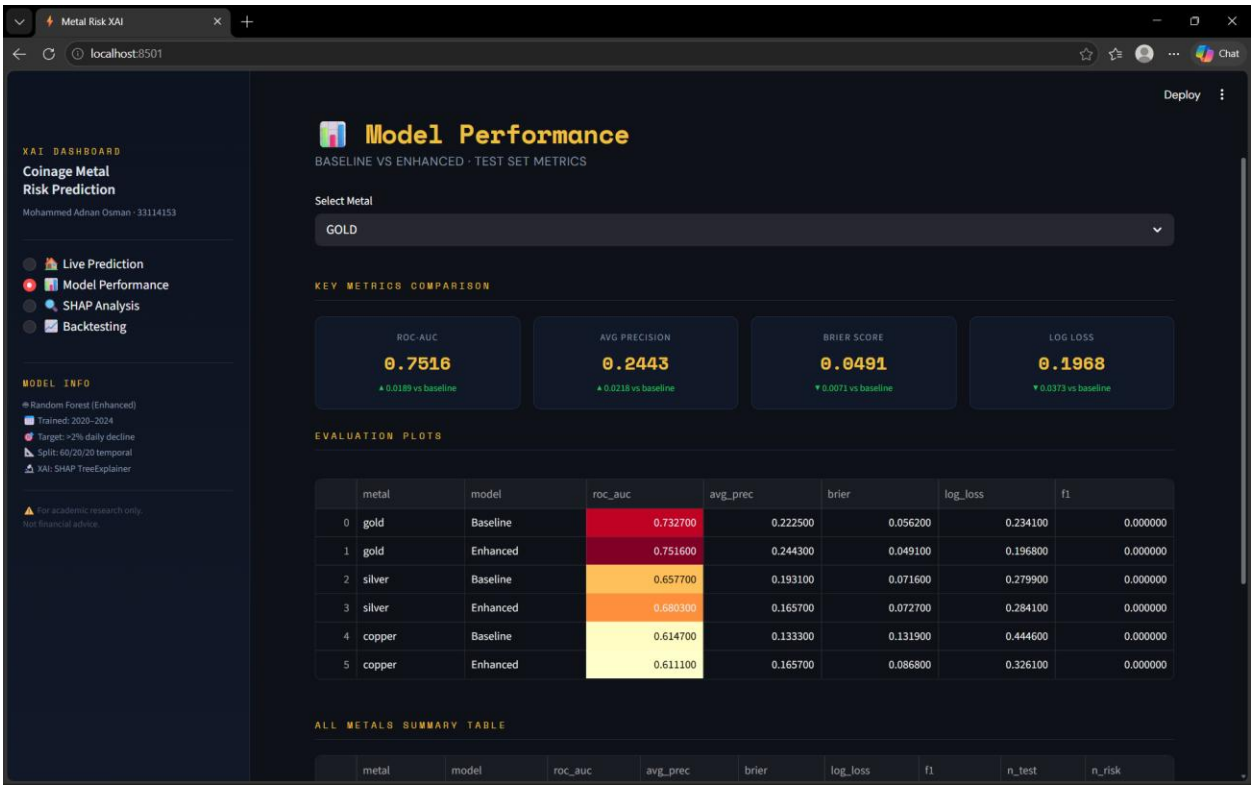


Figure 40: Live downside risk scores and SHAP drivers.



	metal	model	roc_auc	avg_prec	brier	log_loss	f1
0	gold	Baseline	0.732700	0.222500	0.056200	0.234100	0.000000
1	gold	Enhanced	0.751600	0.244300	0.049100	0.196800	0.000000
2	silver	Baseline	0.657700	0.193100	0.071600	0.279900	0.000000
3	silver	Enhanced	0.680300	0.165700	0.072700	0.284100	0.000000
4	copper	Baseline	0.614700	0.133300	0.131900	0.444600	0.000000
5	copper	Enhanced	0.611100	0.165700	0.086800	0.326100	0.000000

	metal	model	roc_auc	avg_prec	brier	log_loss	f1	n_test	n_risk

Figure 41: Gold model performance summary.

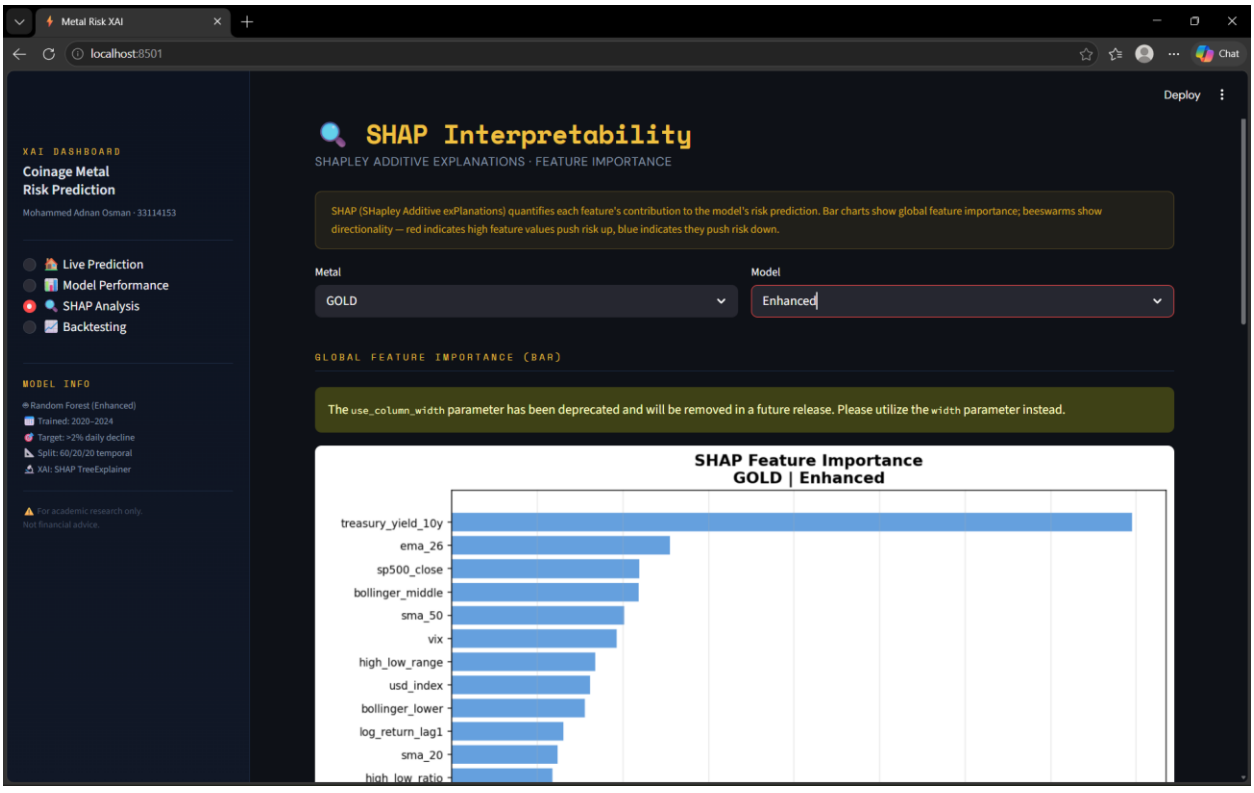


Figure 42: Gold baseline SHAP feature importance.

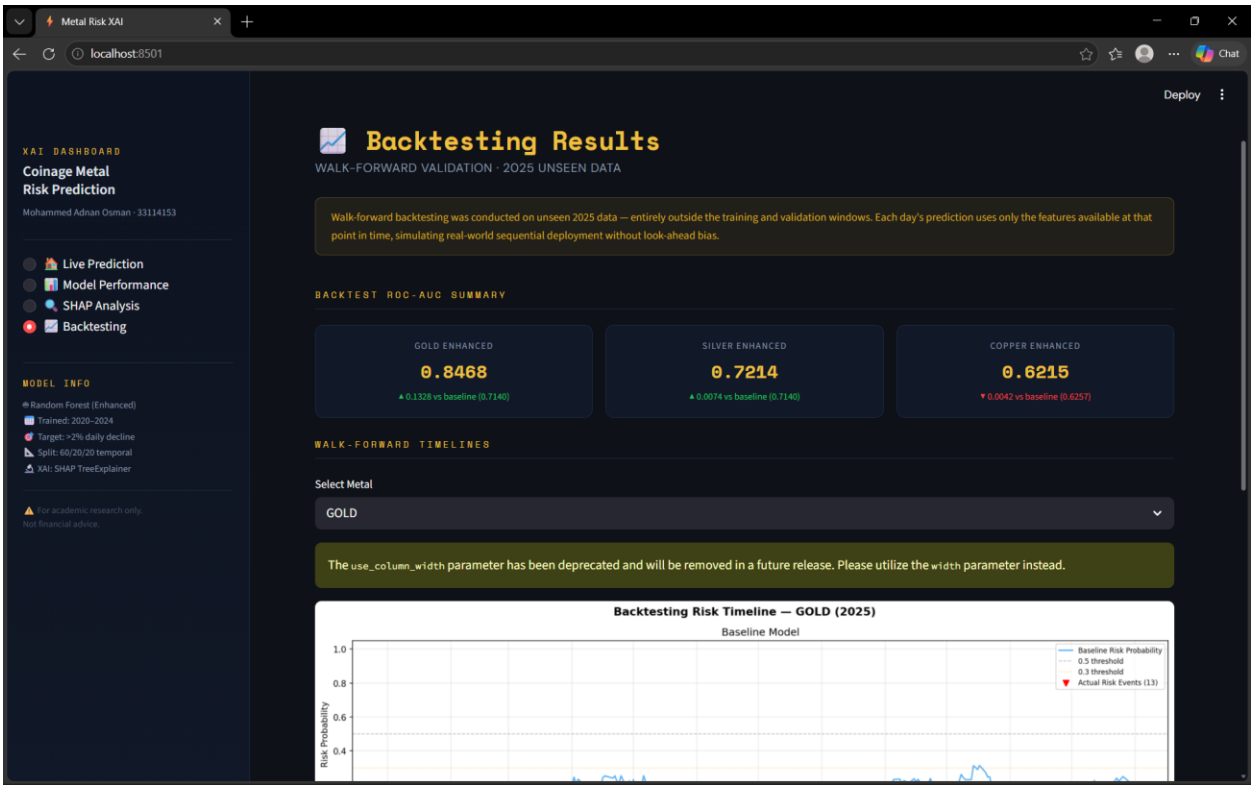


Figure 43: Walk-forward backtesting results and gold timeline.

4.6 System Requirements

Table 2: System requirements summary

Requirement	Specification
Programming Language	Python 3.11
Database	PostgreSQL 15+ (local instance via pgAdmin)
ML Framework	Scikit-learn 1.3+
Explainability	Shap 0.44+
NLP Model	FinBERT via Hugging Face transformers library
Data Source (Price)	yFinance (Yahoo Finance API)
Data Source (News)	NewsAPI (financial news headlines)
Version Control	Git / GitHub
Credential Management	.env file via python-doteny

5 Results and Analysis

5.1 Dataset Overview and Composition

The dataset contains 4,525 trading-day observations across gold, silver, and copper from January 2020 to December 2024. Price and macroeconomic data were collected using yFinance and stored in a PostgreSQL database, while technical indicators were generated using a sliding window approach. The target variable includes 331 downside risk events, defined as next-day price declines greater than 2 percent, which represents 7.3 percent of the sample. This level of imbalance justified the use of class weighting during model training.

FinBERT-based sentiment features were available for only 31 trading days because of NewsAPI free-tier limits. For the remaining days, missing sentiment values were filled using forward-fill imputation. This limited coverage is an important constraint and is discussed further in later chapters because it reduces the expected benefit of sentiment features.

5.2 Model Performance on Test Set

Table 3 presents the performance of the baseline and enhanced Random Forest models across all three metals on the held-out test set, evaluated using ROC-AUC, Brier Score, Log Loss, and F1-score.

Table 3: Test Set Performance: Baseline vs Enhanced Models

Metal	Baseline ROC-AUC	Enhanced ROC-AUC	Baseline Brier	Enhanced Brier	Baseline AP	Enhanced AP
Gold	0.733	0.752	0.056	0.049	0.223	0.244
Silver	0.658	0.680	0.072	0.073	0.193	0.166
Copper	0.615	0.611	0.132	0.087	0.133	0.166

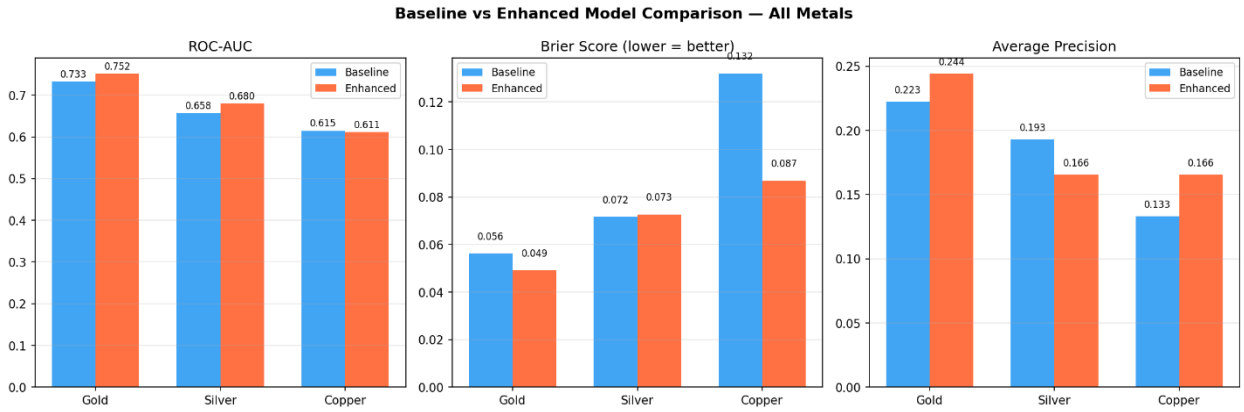


Figure 21: Baseline and enhanced model comparison.

The gold enhanced model achieved the most consistent improvement across metrics. ROC-AUC increased from 0.733 to 0.752, Brier Score improved from 0.056 to 0.049, and Average Precision increased from 0.223 to 0.244. These improvements across all three measures indicate that sentiment integration provided meaningful incremental value for gold, improving both the model's ability to rank risk days correctly and the accuracy of its predicted risk probabilities.

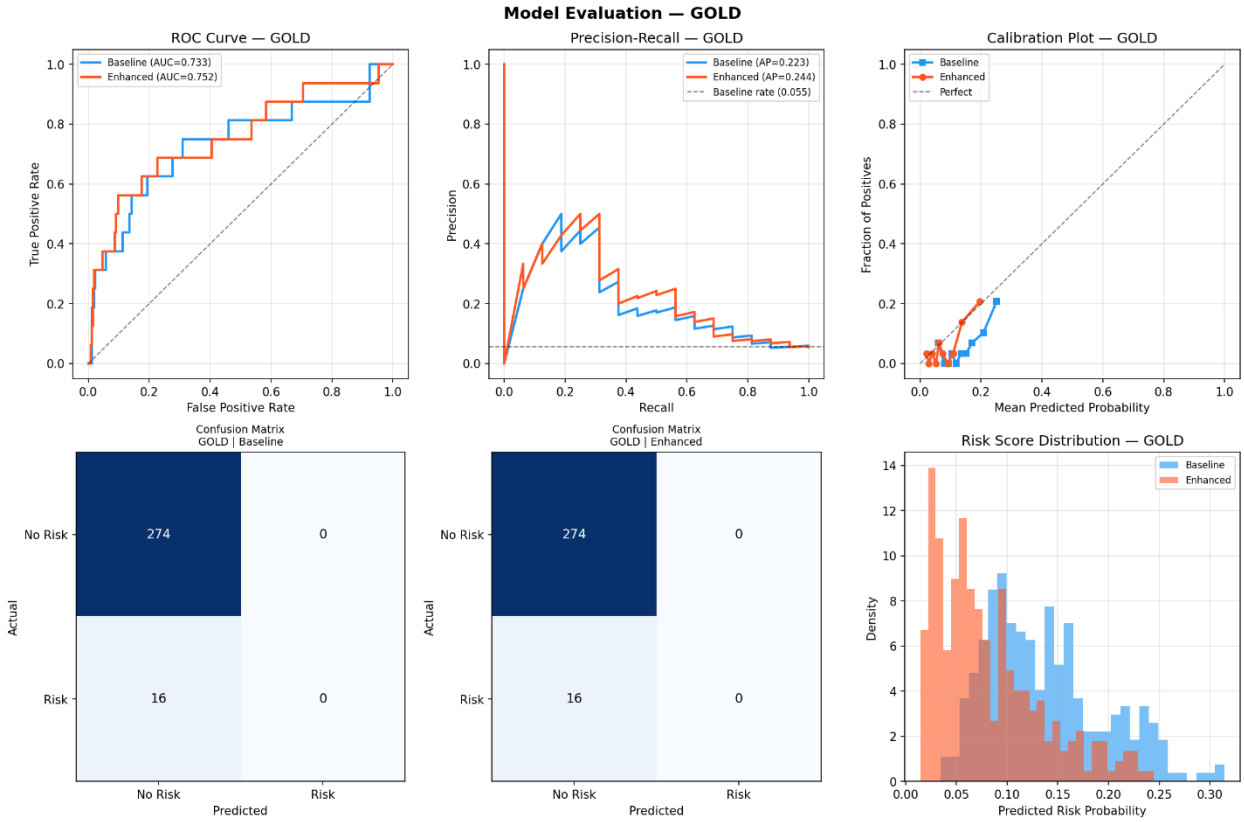


Figure 22: Gold model evaluation plots.

Silver followed a mixed pattern, as shown in Figure 23. The ROC-AUC improved from 0.658 to 0.680 with sentimental features, and the Brier Score showed negligible change. However, Average Precision declined from 0.193 to 0.166, indicating that while the enhanced model ranks risk days more accurately across all thresholds, its precision at high recall levels worsened.

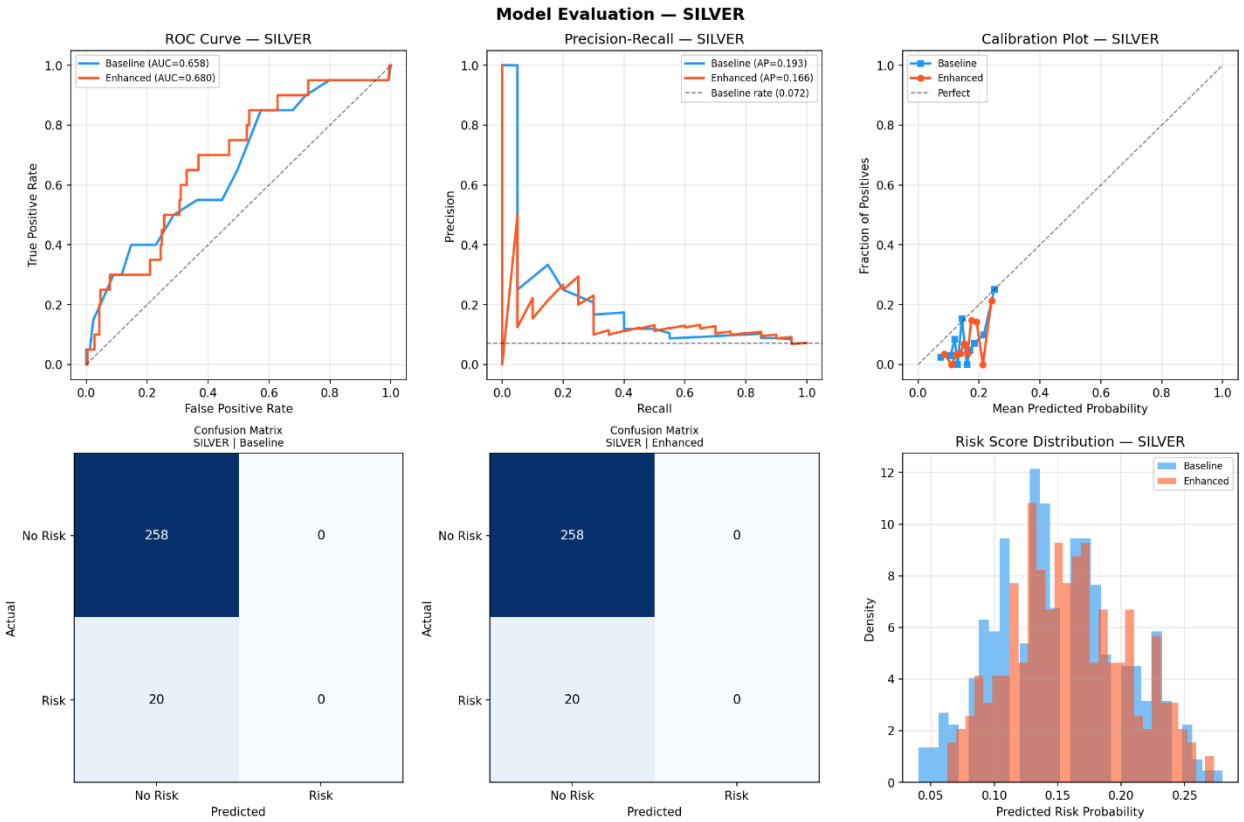


Figure 23: Silver model evaluation plots.

Copper's results show a slight drop in ROC-AUC from 0.615 to 0.611. However, the Brier Score improved sharply from 0.132 to 0.087, and Average Precision rose from 0.133 to 0.166. The risk score distribution also became more concentrated, with the baseline model spreading probabilities more widely and the enhanced model producing lower-risk predictions in a narrower range. This tighter spread helps explain the better Brier Score, since the enhanced model is less likely to assign high risk to non-event days. The unchanged ROC-AUC suggests that ranking performance stayed broadly the same.

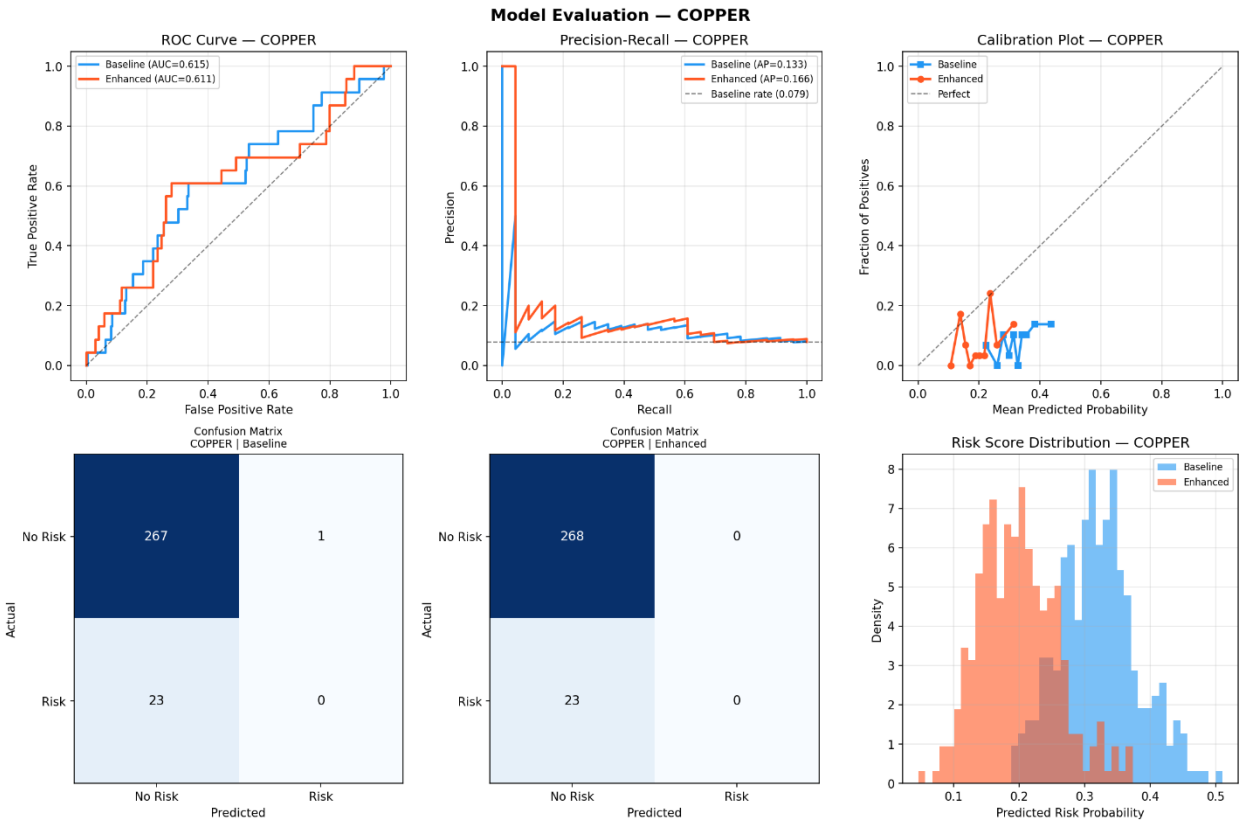


Figure 24: Copper model evaluation plots.

5.3 SHAP Feature Importance Analysis

SHAP TreeExplainer was applied to baseline and enhanced models across all three metals, generating global feature importance bar charts and beeswarm summary plots on the test set.

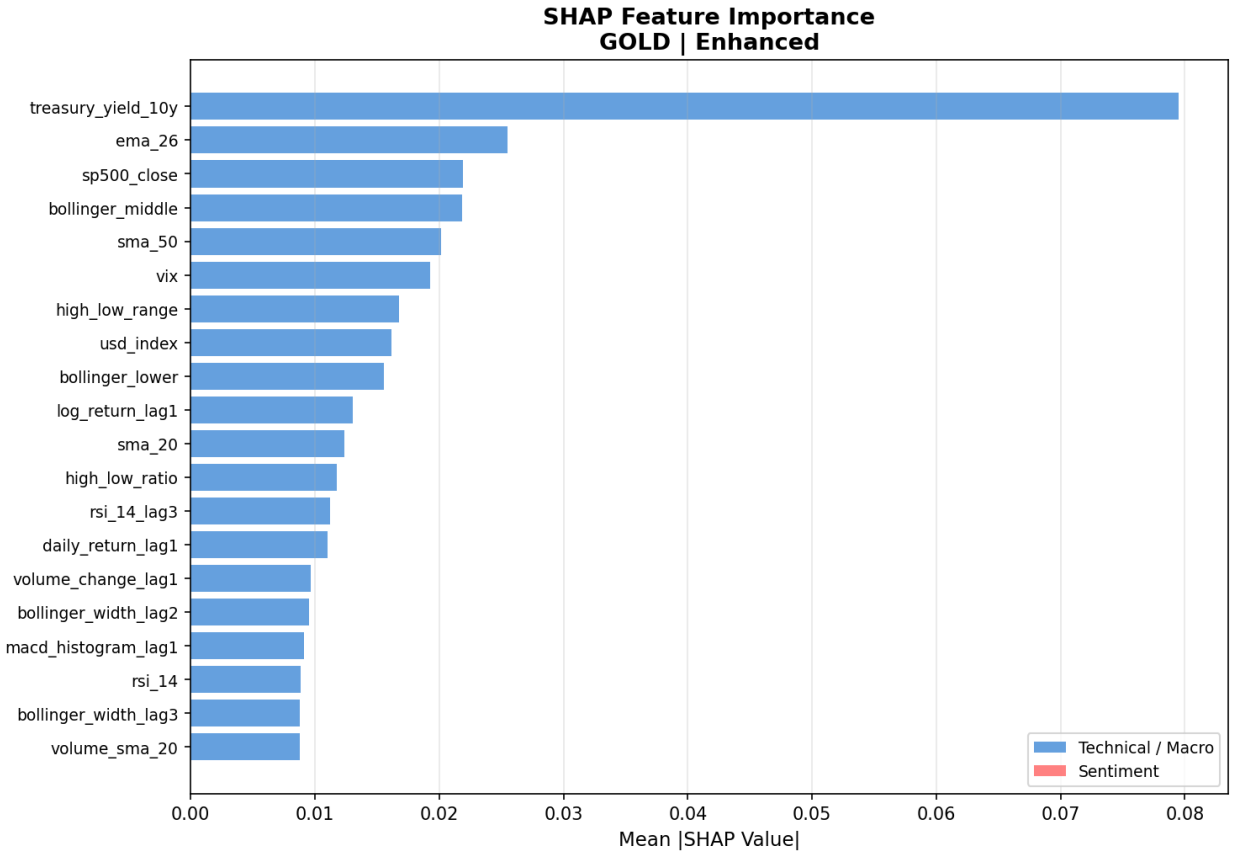


Figure 25: Gold baseline SHAP feature importance.

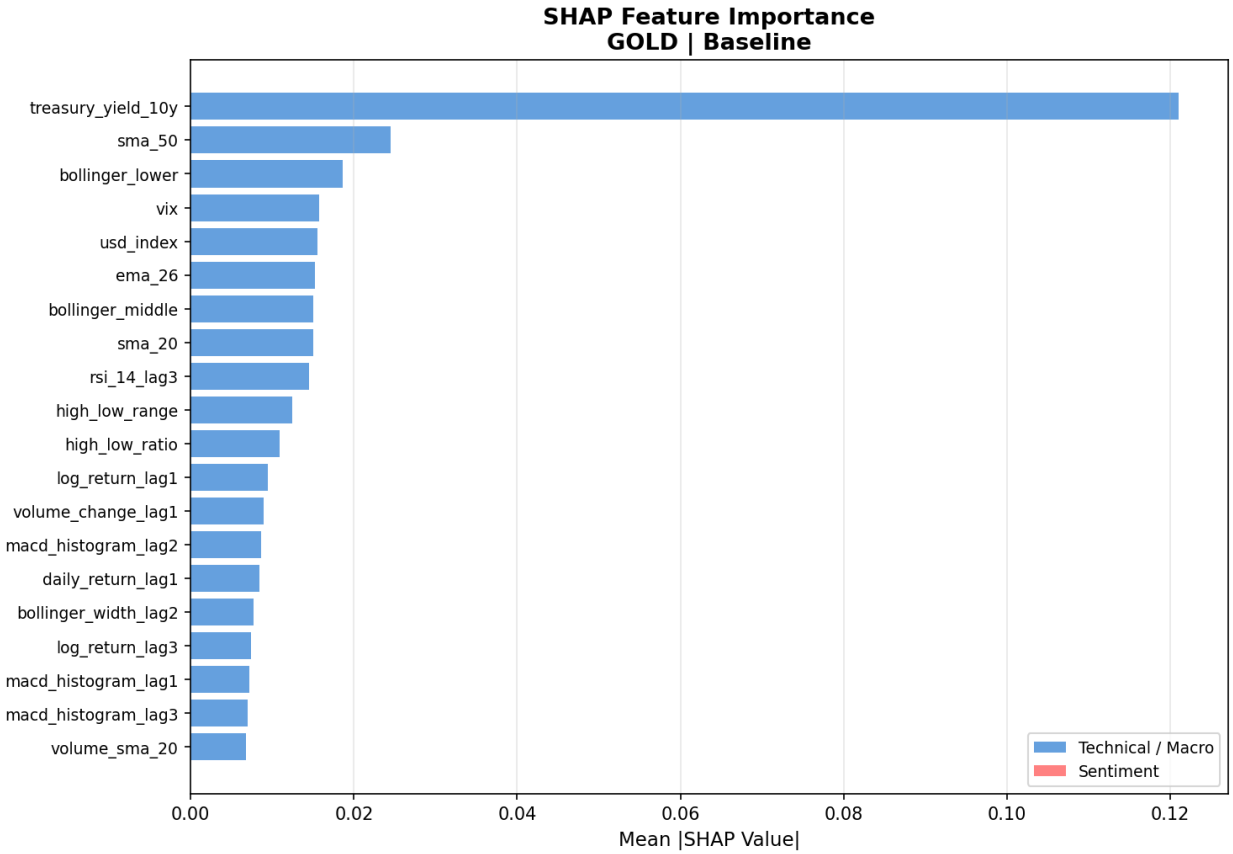


Figure 26: Gold baseline SHAP feature importance.

As shown in the analysis, the 10-year US Treasury yield (treasury_yield_10y) is the dominant predictive feature for gold in both models. In the baseline model, it has a mean absolute SHAP value of 0.12, which is substantially higher than the second-ranked variable (sma_50) at 0.025. This dominance continues in the enhanced model where the yield retains the top ranking with a SHAP value of 0.08. Its reduced importance in the enhanced model reflects how predictive weight is redistributed across the broader feature set when sentiment variables are added rather than any change in the underlying predictive relationship.

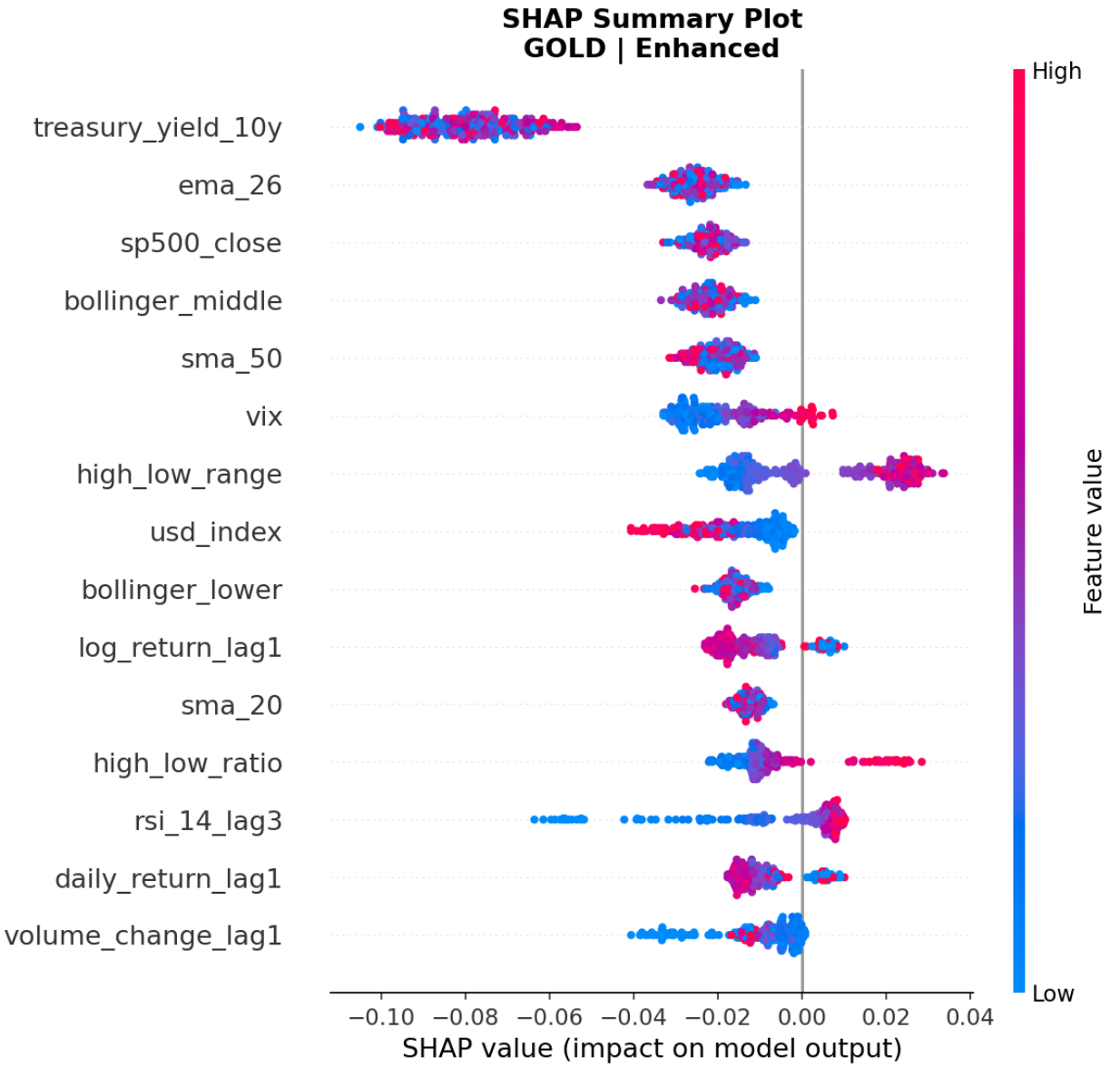


Figure 27: SHAP beeswarm summary plot: Gold Enhanced model.

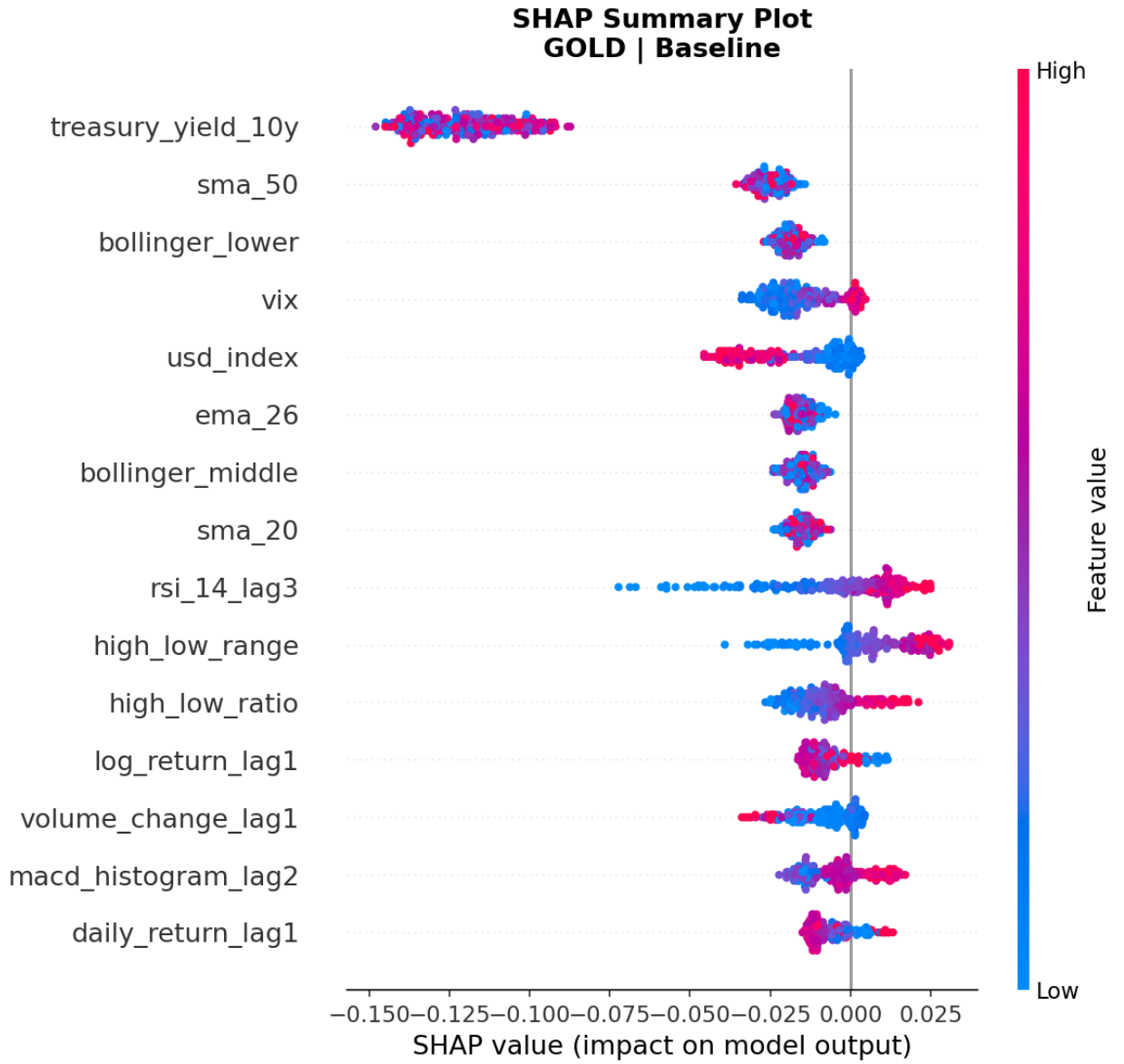


Figure 28: Gold baseline SHAP beeswarm summary.

The beeswarm plot for the gold baseline gives directional insight into how the Treasury yield affects predictions. Most 10-year Treasury yield values lie to the left of zero, suggesting that higher yields are generally linked to lower predicted downside risk. The US Dollar Index shows the opposite pattern, with high values appearing to the right of zero, which is consistent with dollar strength increasing predicted downside risk for gold. The VIX is more spread out around zero, indicating a more complex and non-linear effect that depends on market conditions.

For silver, the VIX is the dominant feature, followed by the 10-year Treasury yield and the S&P 500 return. Volume-related features also rank more prominently for silver than they do for gold.

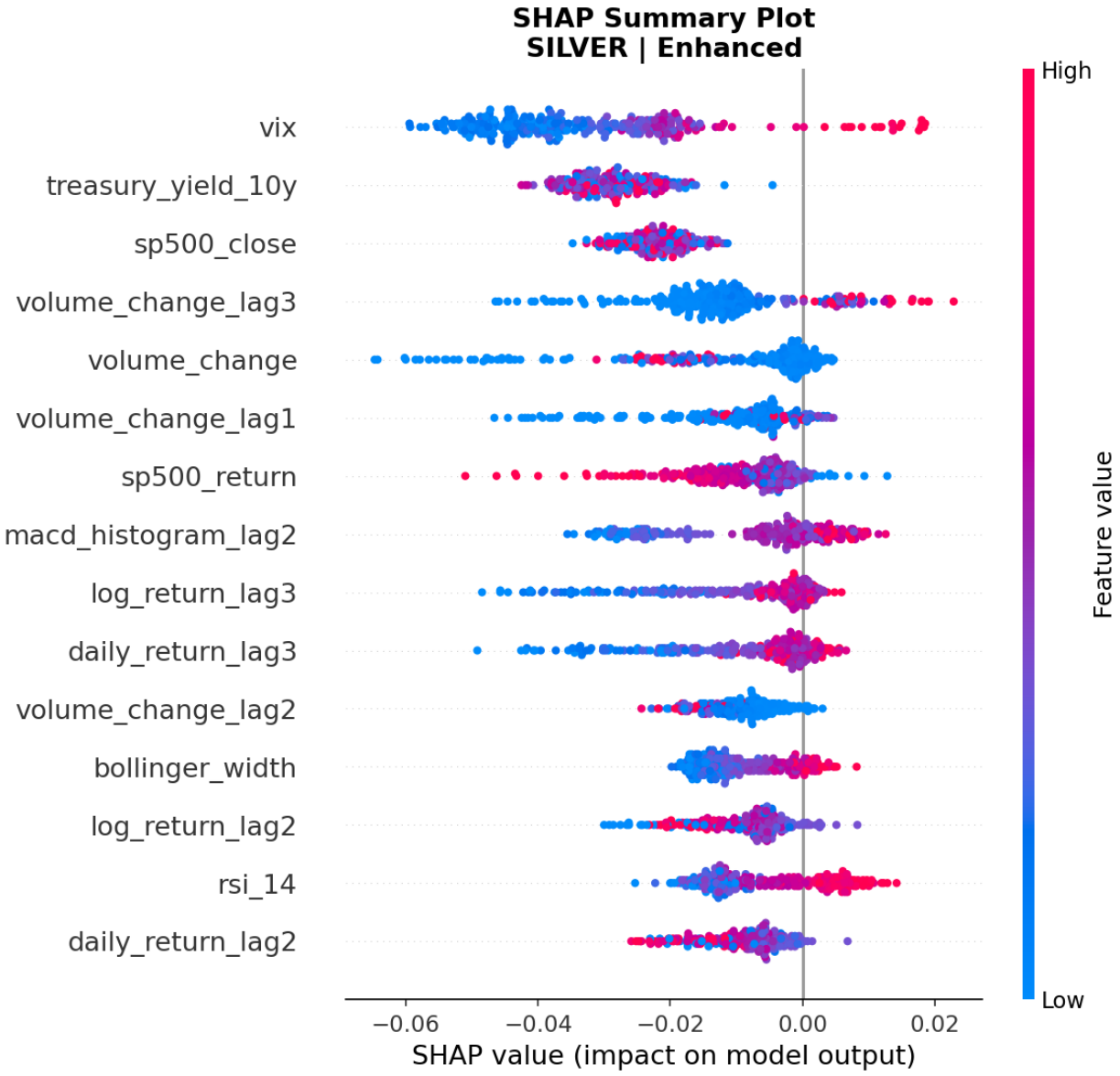


Figure 29: SHAP beeswarm summary plot: Silver Enhanced model.

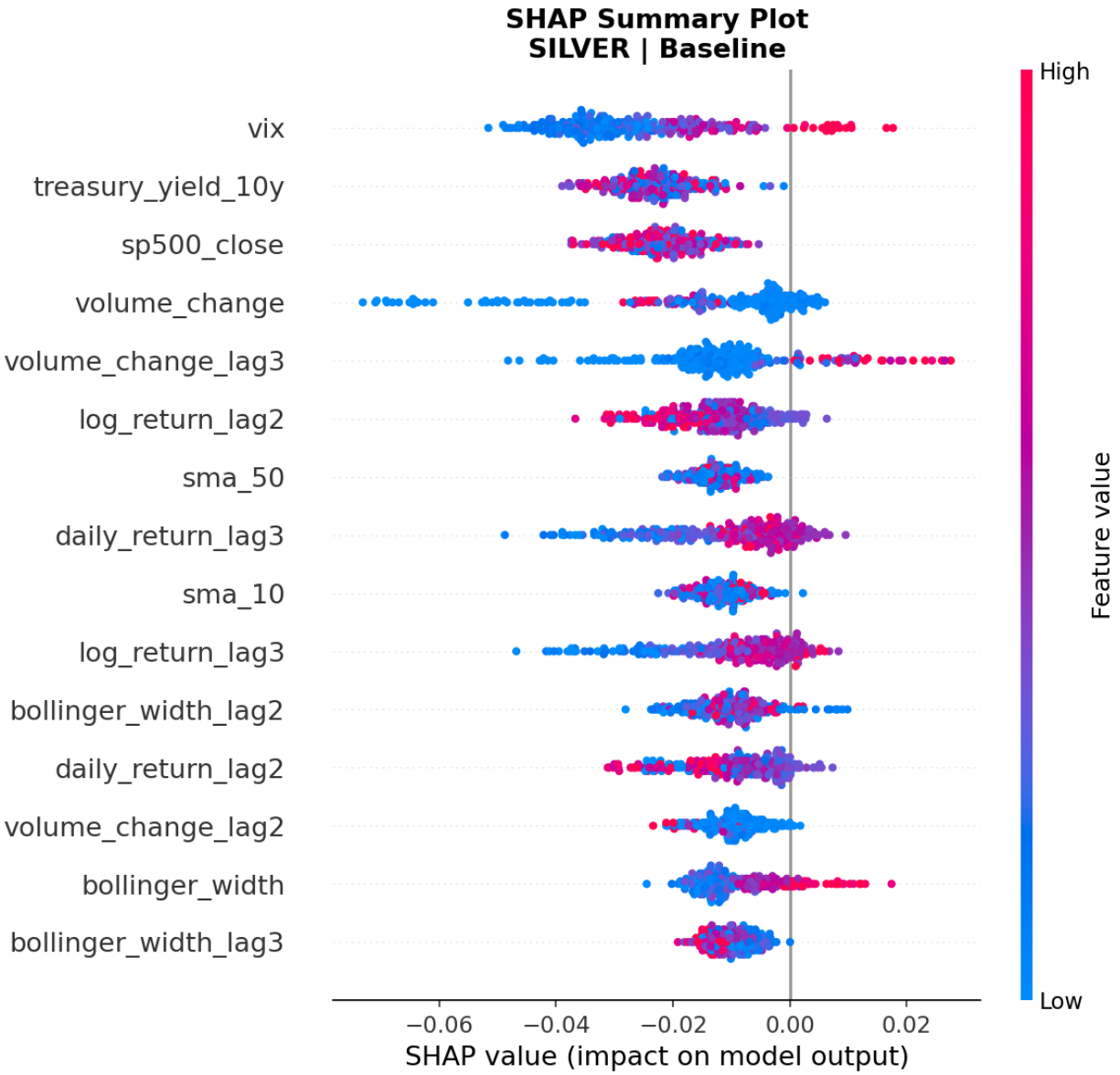


Figure 30: SHAP beeswarm summary plot: Silver Baseline model.

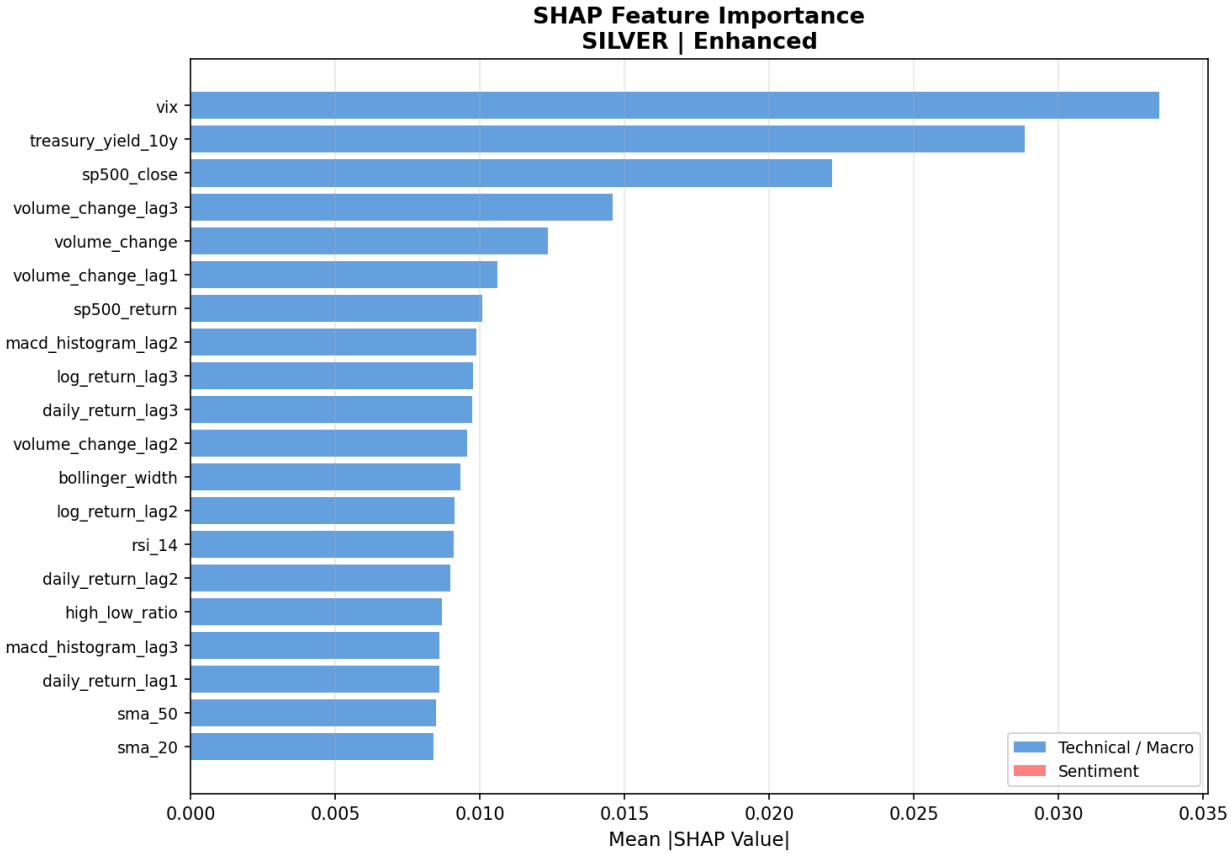


Figure 31: SHAP feature importance bar chart: Silver Enhanced model.

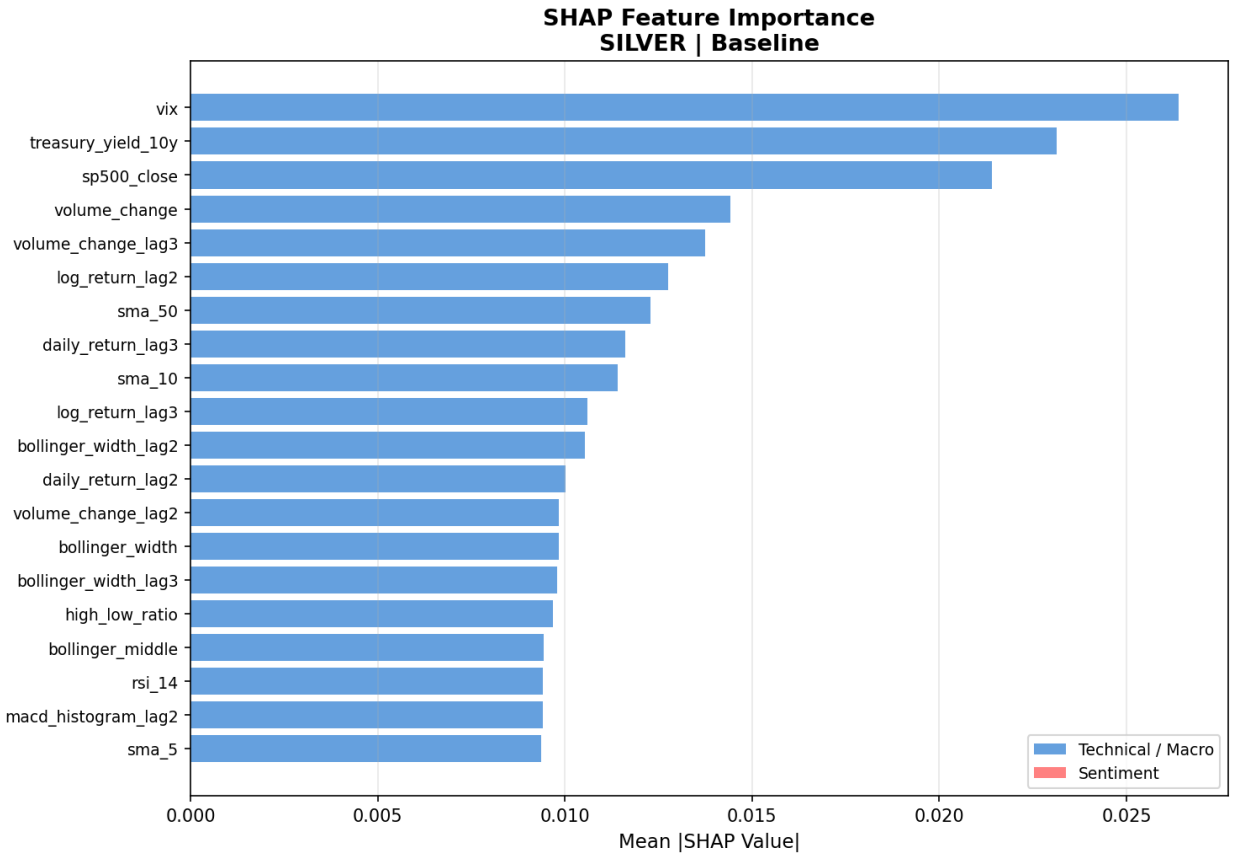


Figure 32: SHAP feature importance bar chart: Silver Baseline model.

Copper's SHAP profile is different from the precious metals. The S&P 500 return is the most important baseline feature, followed by the S&P 500 close, the three-day volume change, and the high-low range. This is sensible because copper is closely tied to global growth, so broader equity market movements reflect the same macroeconomic sentiment that drives copper demand. The beeswarm plot shows that higher S&P 500 returns are associated with negative SHAP values, meaning strong equity performance reduces predicted copper downside risk, which matches copper's procyclical nature.

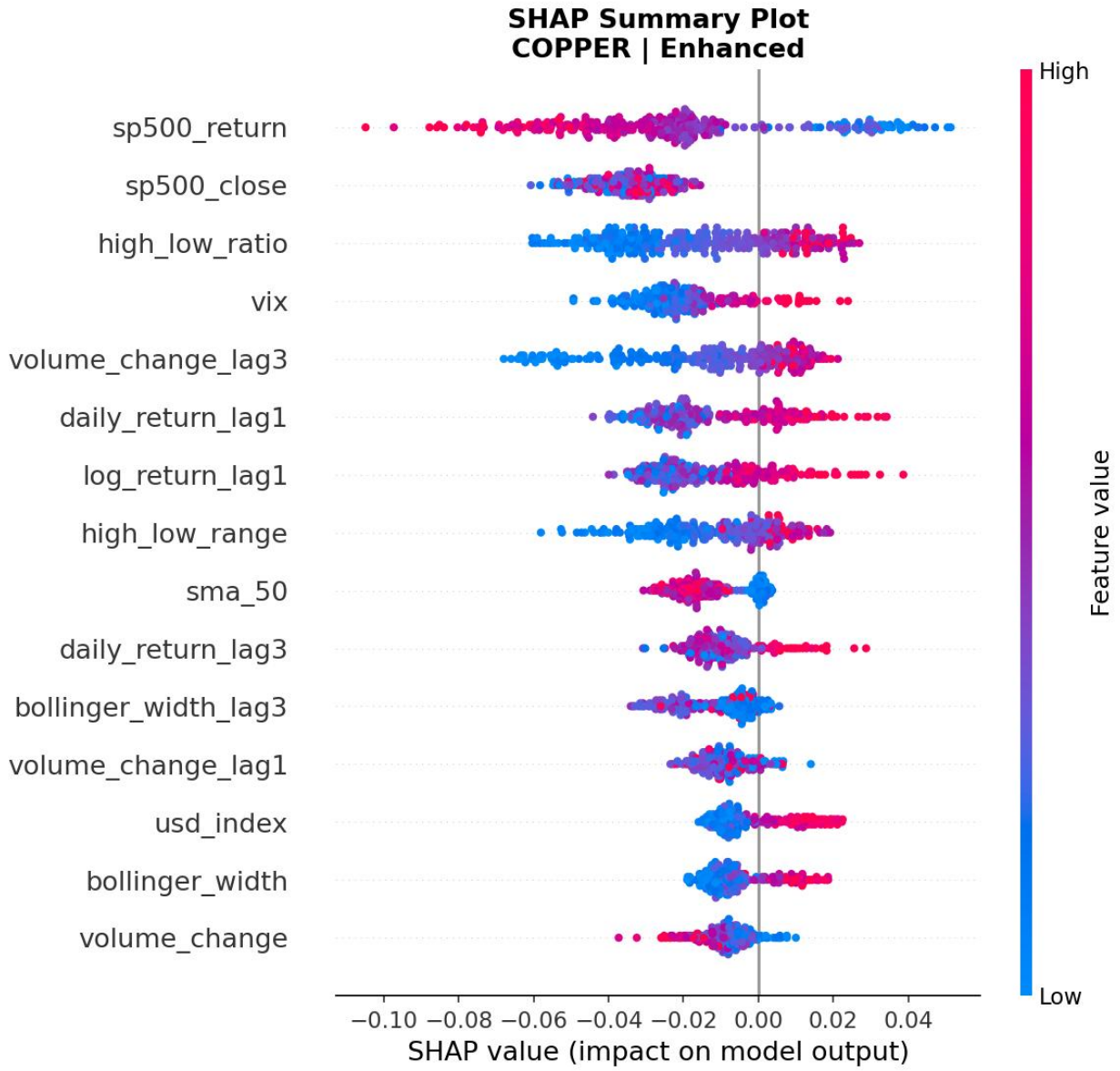


Figure 33: SHAP beeswarm summary plot: Copper Enhanced model.

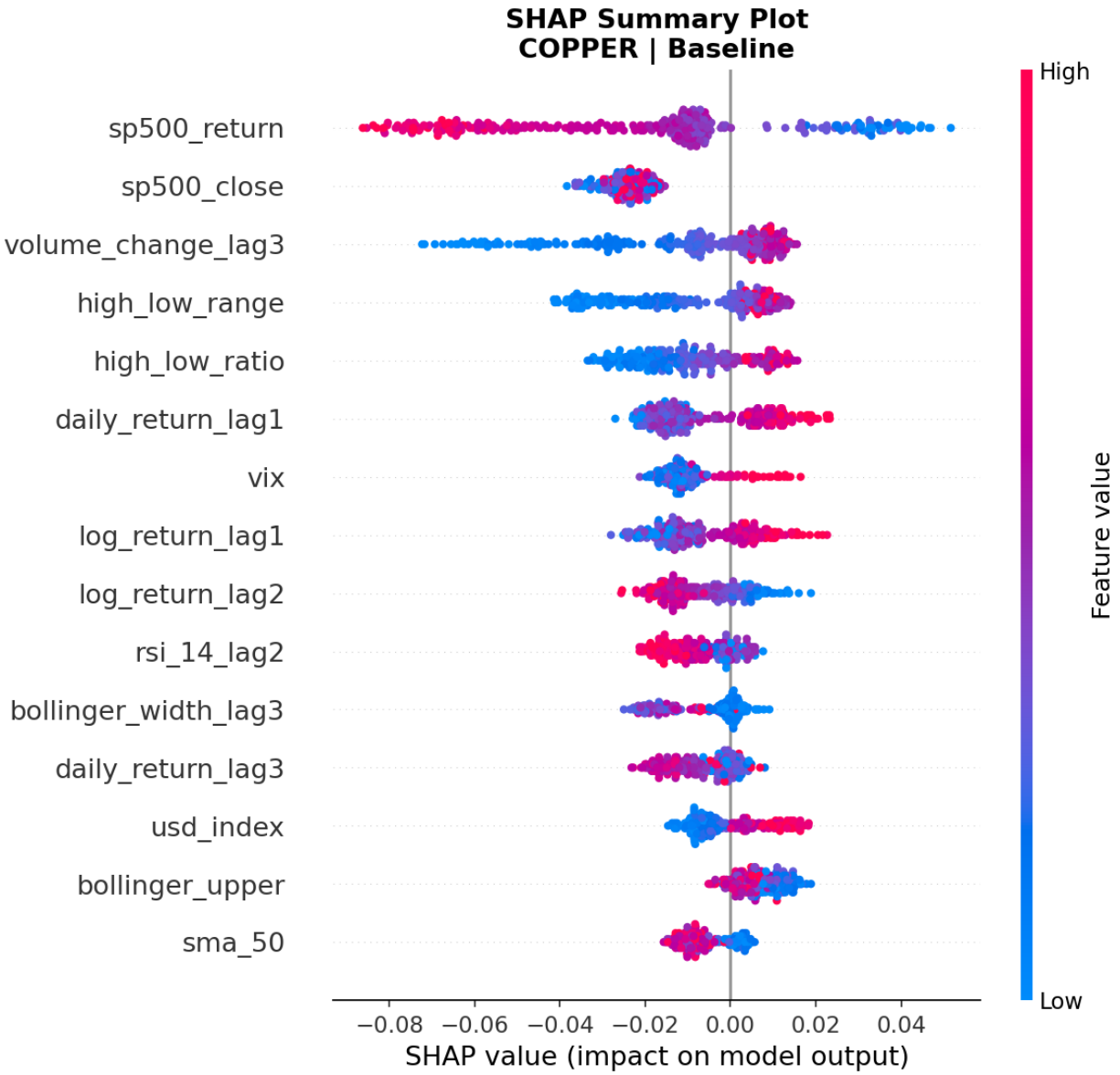


Figure 34: SHAP beeswarm summary plot: Copper Baseline model.

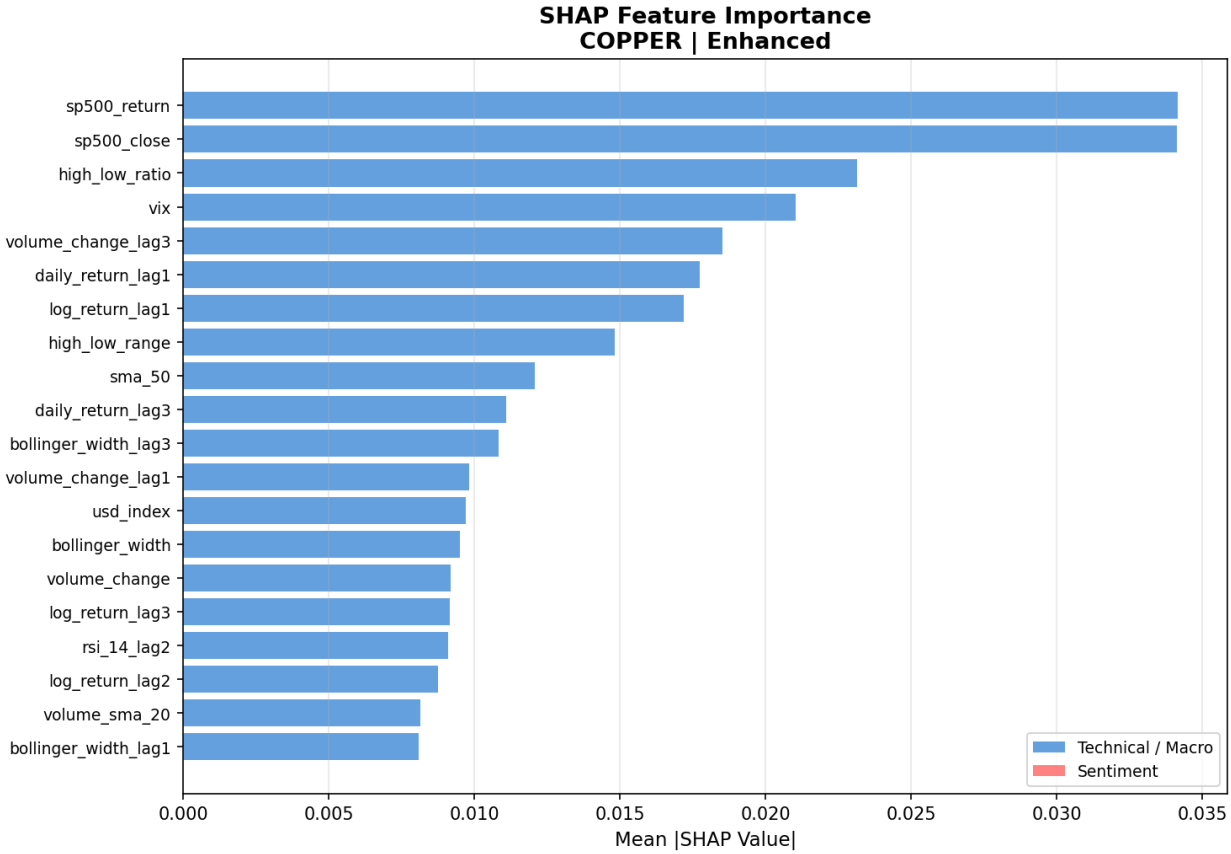


Figure 35: SHAP feature importance bar chart: Copper Enhanced model.

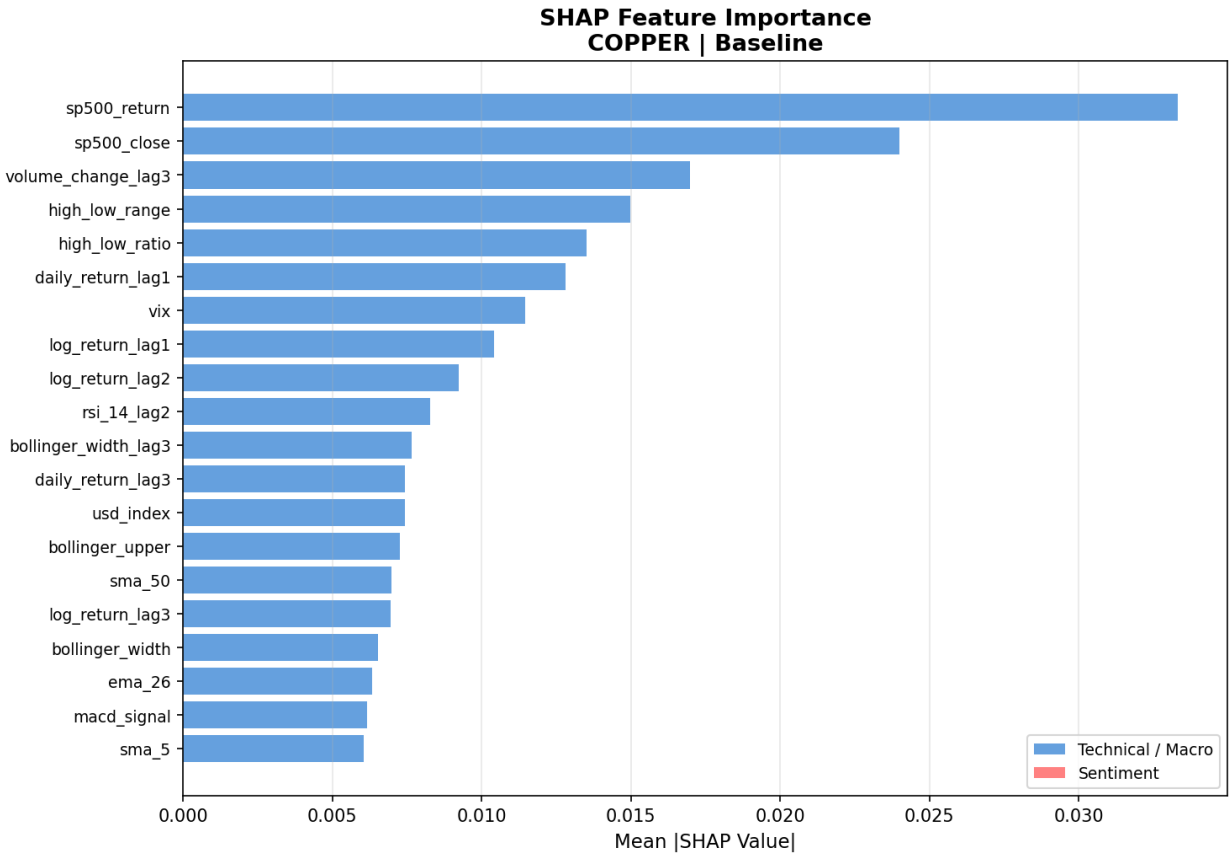


Figure 36: SHAP feature importance bar chart: Copper Baseline model.

5.4 Sentiment Feature Impact

Gold showed consistent gains from sentiment integration. ROC-AUC rose from 0.733 to 0.752, Brier Score improved from 0.056 to 0.049, and Average Precision increased from 0.223 to 0.244. It was the only metal where sentiment improved every metric, suggesting that FinBERT adds useful information for gold because its price is strongly influenced by macroeconomic and geopolitical news.

Silver was more mixed. ROC-AUC improved from 0.658 to 0.680, but Average Precision fell from 0.193 to 0.166. This suggests that the enhanced model ranks risk days better overall, but is less precise when identifying the most important risk periods. A possible reason is silver's dual role as both a safe-haven asset and an industrial metal, which can produce conflicting sentiment signals.

Copper showed the clearest calibration benefit. ROC-AUC fell slightly by 0.004, so there was no meaningful improvement in ranking ability. However, Brier Score improved sharply from 0.132 to 0.087, which is the largest calibration gain in the study. This suggests that

sentiment helps copper mainly by reducing overconfident high-risk predictions on non-event days, rather than by improving its ability to rank risk days.

5.5 Back testing on Unseen 2025 Data

Walk-forward back testing was run on data from January 2025 to January 2026, which was fully excluded from training and validation.

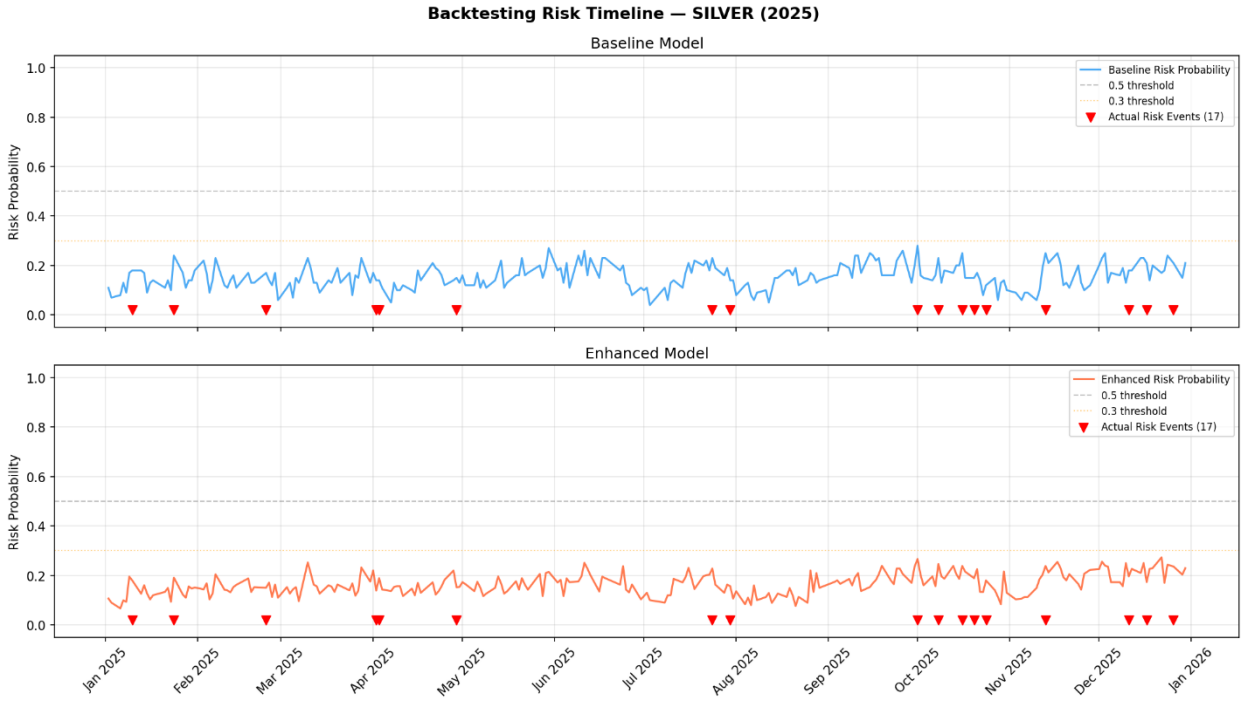


Figure 37: Silver backtesting risk timeline.

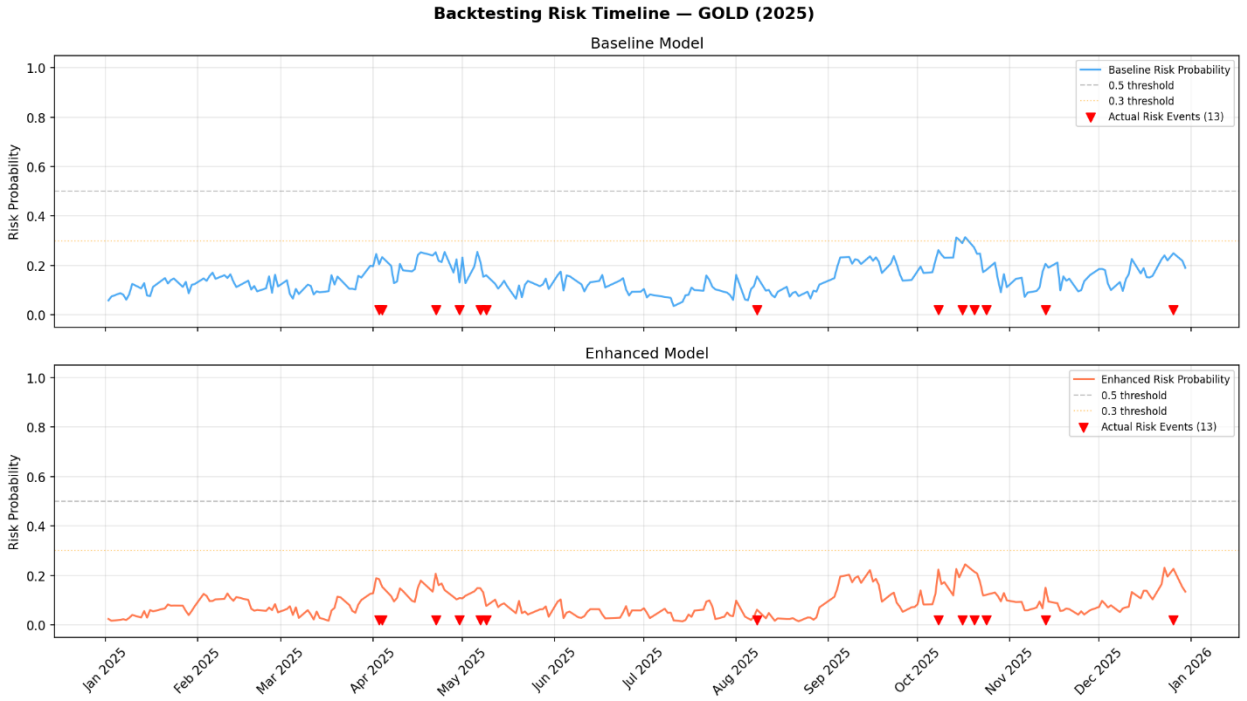


Figure 38: Gold backtesting risk timeline.

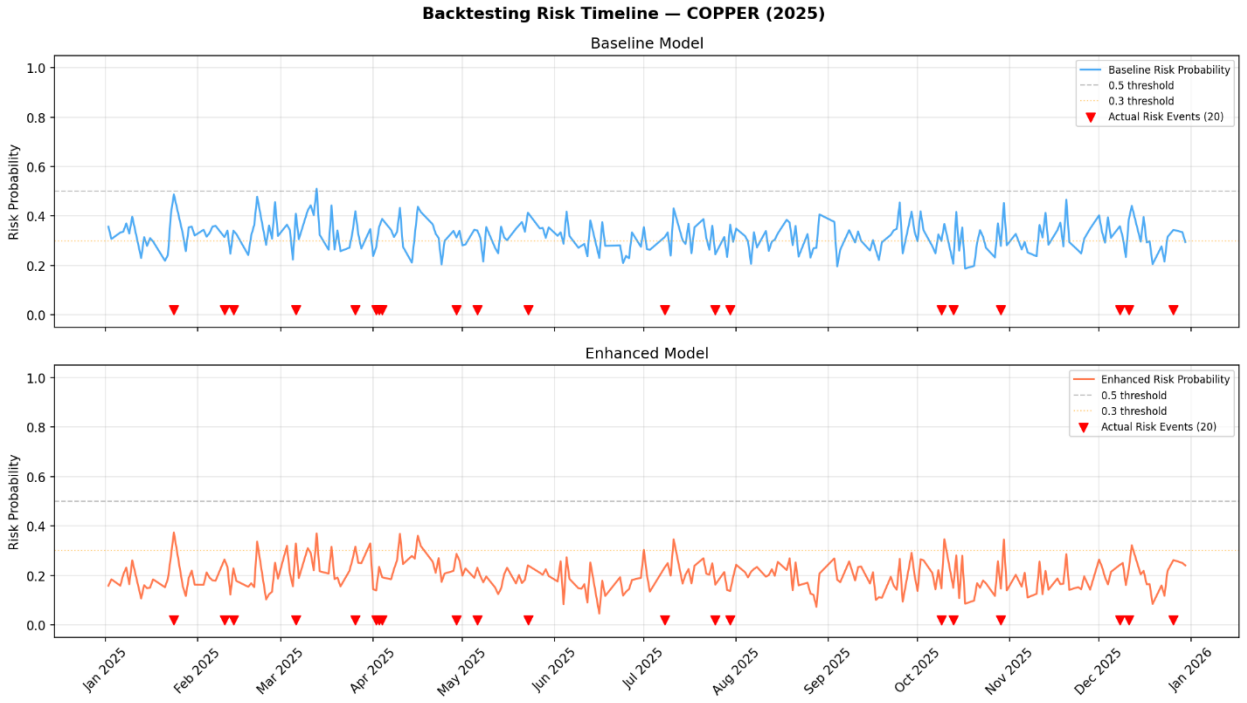


Figure 39: Copper backtesting risk timeline.

Several patterns emerge from the backtesting timelines. For gold, the baseline model mostly stays between 0.05 and 0.25 during 2025, with a clear increase in October and November that lines up with Federal Reserve commentary on rates and dollar strength. The enhanced model is more conservative, with probabilities mostly between 0.02 and 0.15,

which reflects its better calibration. The 13 actual gold risk events, especially those in spring and autumn, tend to line up with higher probabilities from both models, suggesting a real signal in the unseen data.

Silver shows a similar pattern, but the baseline and enhanced timelines are closer together, with risk scores generally staying below 0.30. The 17 actual silver risk events are spread across the year, with a concentration in October and November that matches the gold pattern. This is consistent with the way precious metals often move together during periods of macroeconomic stress.

Copper shows the sharpest difference between the two models. The baseline model produces much wider probabilities, often between 0.20 and 0.50, with several spikes near or above the decision threshold. The enhanced model compresses these predictions into a narrower 0.10 to 0.35 range. That narrower spread helps explain the strong Brier Score improvement, since the enhanced model is less prone to assigning overly high risk on non-event days.

6 Discussion and Conclusion

6.1 Discussion

The central research question asked whether a Random Forest classifier with FinBERT sentiment could accurately predict daily downside risk events in gold, silver, and copper while also producing interpretable probability-based risk scores and SHAP explanations. The results suggest a nuanced answer: the approach shows meaningful predictive ability for gold and silver, and it produces well-calibrated probabilities across all three metals.

The SHAP explanations are also broadly consistent with established financial theory, which supports the model's interpretability. However, the practical contribution of sentiment features appears limited by the sparse historical coverage of the sentiment dataset. Overall, the framework is promising, but its strongest value lies in calibrated risk estimation and explainability rather than in sentiment alone driving performance gains.

6.1.1 Model Performance in Relation to Literature

The gold enhanced model's ROC-AUC values of 0.752 on the test set and 0.847 on the backtest are consistent with results reported in similar financial risk studies. Prior research by Fischer and Krauss reported ROC-AUC values in the 0.71 to 0.78 range for Random Forest models on equities, while a FinBERT-XGBoost study achieved an AUC of 0.748 using combined technical and sentiment features. This places the gold results comfortably

within the established performance band and supports the soundness of the experimental design.

The improvement from the enhanced model is modest but consistent for both gold and silver in ROC-AUC terms. The calibration gains, especially the stronger Brier Score improvement for copper, also align with Li et al. (2020), who found that sentiment integration often improves probability calibration even when ranking performance changes little. Taking together, these findings extend earlier literature into the coinage metals context and provide new empirical evidence for an area that has received limited attention in explainable AI research.

6.1.2 SHAP Interpretability and Alignment with Financial Theory

The SHAP analysis supports economically intuitive relationships. For gold, the 10-year Treasury yield is the dominant driver with a negative contribution, while silver is influenced mainly by the VIX and Treasury yields, and copper is led by S&P 500 returns, consistent with its role as a procyclical industrial metal.

A key limitation is that sentiment features do not appear in the top 20 SHAP rankings. This seems to reflect sparse sentiment coverage rather than weak signal quality, so the objective of producing SHAP explanations that clearly incorporate sentiment was only partially met. With a more complete sentiment dataset, those features would likely become more visible in global importance rankings and individual predictions, in line with Dhole (2024) and Gu et al. (2024).

6.1.3 Back testing and Real-World Applicability

The back testing results on the 2025 unseen data provide encouraging evidence of generalization, particularly for gold where the enhanced model achieved a ROC-AUC of 0.847. This qualitative validation is consistent with the approach recommended by Giudici et al. (2024), who argue that SHAP-based case analysis of back test predictions provides important supplementary evidence of model reliability beyond quantitative metrics alone.

6.1.4 Limitations

Several limitations must be acknowledged. First, sentiment data is a significant constraint. The NewsAPI free tier limits access to recent headlines, meaning sentiment features were available for less than 3% of training observations.

Second, class imbalance remains a fundamental challenge. The low F1-scores at default thresholds indicate that the models do not generate actionable binary signals without

adjustment. Any practical application would require calibrating the operating threshold based on a specific tolerance for false positives and false negatives.

Third, financial markets are non-stationary, meaning models trained on historical data may not perform well in unprecedented conditions. The strong backtest performance in 2025 is encouraging, but that period shares macroeconomic characteristics with the training window. Consequently, performance in a structurally different macroeconomic regime cannot be guaranteed.

6.2 Conclusion

This project developed an explainable machine learning system to predict short-term downside risk in coinage metal markets. It integrated technical price indicators, macroeconomic features, and sentiment analysis to generate probability-based risk scores supported by SHAP explanations. The system was implemented as a Python pipeline connected to a PostgreSQL database, covering data collection, feature engineering, model training, and backtesting.

The primary limitation is restricted sentiment coverage, which prevented sentiment features from contributing visibly to SHAP global importance rankings despite producing measurable metric improvements. While this does not invalidate the findings, it limits the conclusions that can be drawn regarding the interpretability of sentiment-driven predictions.

The project objectives were met, and the findings add to existing research on explainable AI in commodity markets. This study demonstrates that interpretable machine learning frameworks can produce both practically useful risk scores and meaningful feature insights for coinage metal risk prediction.

Future work should prioritize three areas. First, expanding sentiment coverage with a professional data source would allow a more definitive assessment of FinBERT's contribution. Second, threshold calibration studies should identify operating points that balance precision and recall for specific risk management needs. Third, incorporating recurrent neural network components such as LSTM layers would better capture temporal dependencies that the current feature approach only partially addresses.

References

Araci, D. (2019) *FinBERT: Financial sentiment analysis with pre-trained language models*. arXiv preprint arXiv:1908.10063. Available at: https://arxiv.org/abs/1908.10063 (Accessed: 10 November 2025).

Bollen, J., Mao, H. and Zeng, X. (2011) 'Twitter mood predicts the stock market', *Journal of Computational Science*, 2(1). Available at: https://arxiv.org/pdf/1010.3003 (Accessed: 29 September 2025).

Danie, A. et al. (2025) 'Model-agnostic explainable artificial intelligence methods in finance: a systematic review', *Artificial Intelligence Review*, 58. Available at: https://link.springer.com/article/10.1007/s10462-025-11215-9 (Accessed: 12 March 2026).

Doshi-Velez, F. and Kim, B. (2018) 'Towards a rigorous science of interpretable machine learning', *arXiv preprint*. Available at: https://arxiv.org/pdf/1702.08608 (Accessed: 2 October 2025).

Fischer, T. and Krauss, C. (2018) 'Deep learning with long short-term memory networks for financial market predictions', *European Journal of Operational Research*, 270(2). Available at: https://iranarze.ir/wp-content/uploads/2019/01/E10789-IranArze.pdf (Accessed: 17 October 2025).

Giudici, P., Piergallini, A., Recchioni, M.C. and Raffinetti, E. (2024) 'Explainable Artificial Intelligence methods for financial time series', *ScienceDirect*. Available at: https://www.sciencedirect.com/science/article/pii/S037843712400685X (Accessed: 12 November 2025).

Gu, W. et al. (2024) 'Predicting stock prices with FinBERT-LSTM: integrating news sentiment analysis', *Proceedings of the 2024 8th International Conference on Computer Science and Artificial Intelligence*. Available at: https://arxiv.org/pdf/2407.16150 (Accessed: 14 March 2026).

Jabeur, S.B., Mefteh-Wali, S. and Viviani, J.L. (2024) 'Forecasting gold price with the XGBoost algorithm and SHAP interaction values', *Annals of Operations Research*, 334, pp.679–699. Available at: https://link.springer.com/article/10.1007/s10479-021-04187-w (Accessed: 15 November 2025).

Li, Q., Wang, T. and Wu, J. (2020) 'Financial sentiment analysis for risk prediction using hybrid deep learning', *Expert Systems with Applications*. Available at: https://www.researchgate.net/publication/258821644 (Accessed: 28 October 2025).
Liapis, C.M. and Karanikola, A. (2023) 'Investigating deep stock market forecasting with sentiment analysis', *Entropy*, 25(2), p.219. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC9955765/ (Accessed: 14 March 2026).

Liu, Z., Huang, D., Huang, K., Li, Z. and Zhao, J. (2020) 'FinBERT: a pre-trained financial language representation model for financial text mining', *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. Available at: https://www.ijcai.org/proceedings/2020/0622.pdf (Accessed: 10 November 2025).

Lundberg, S.M. and Lee, S.I. (2017) 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems (NeurIPS)*. Available at: https://arxiv.org/pdf/1705.07874 (Accessed: 2 November 2025).

Mathematics (2025) 'Stock price prediction using FinBERT-enhanced sentiment with SHAP explainability and differential privacy', *Mathematics*, 13(17), p.2747. Available at: https://www.mdpi.com/2227-7390/13/17/2747 (Accessed: 15 March 2026).

Molnar, C. (2022) *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd edn. Independently published. Available at: https://originalstatic.aminer.cn/misc/pdf/Molnar-interpretable-machinelearning_compressed.pdf (Accessed: 9 November 2025).

Monica Hovsepian (2023) 'The benefits and risks of AI in financial services', *Forbes Finance Council*. Available at: https://www.forbes.com/councils/forbesfinancecouncil/2023/12/26/the-benefits-and-risks-of-ai-in-financial-services/ (Accessed: 12 November 2025).

Peng, K. and Yan, G. (2021) 'A survey on deep learning for financial risk prediction', *Quantitative Finance and Economics*, 5(4), pp.716–737. Available at: https://www.aimspress.com/article/doi/10.3934/QFE.2021032 (Accessed: 13 March 2026).

Tuitoek, J. et al. (2025) 'Modelling commodity price co-movement: building on traditional time series models and exploring applications of machine learning algorithms', *Decisions in Economics and Finance*. Available at: https://link.springer.com/article/10.1007/s10203-025-00512-1 (Accessed: 14 March 2026).

Appendix

Model Evaluation Metrics Summary

Table 4: Full evaluation metrics for all six model variants on the held-out test set

Metal	Model	ROC-AUC	Avg Precision	Brier Score	Log Loss	F1	N_test	N_risk
Gold	Baseline	0.7327	0.2225	0.0562	0.2341	0.0	290	16
Gold	Enhanced	0.7516	0.2443	0.0491	0.1968	0.0	290	16
Silver	Baseline	0.6577	0.1931	0.0716	0.2799	0.0	278	20
Silver	Enhanced	0.6803	0.1657	0.0727	0.2841	0.0	278	20
Copper	Baseline	0.6147	0.1333	0.1319	0.4446	0.0	291	23
Copper	Enhanced	0.6111	0.1657	0.0868	0.3261	0.0	291	23